# Macro Portfolio Allocation Model

George Guo

2026-03-23

## Macro Portfolio Allocation Model: A Regime-Based Approach to Asset Allocation

### 1. Introduction

Macroeconomic conditions play a central role in financial markets because changes in inflation, growth, interest rates, and credit conditions tend to affect asset classes in different ways. Equities often perform well during periods of stable growth and contained inflation, while bonds may perform better during economic slowdowns or recessionary environments. Real assets such as gold or commodities may become more attractive when inflation accelerates or when traditional financial assets face pressure. Because different assets respond differently to changing economic conditions, portfolio allocation can potentially be improved by adjusting exposures as the macroeconomic backdrop changes.

Traditional benchmark portfolios such as a 60/40 stock-bond mix are static. They do not respond to shifts in inflation trends, labor market weakness, yield curve inversion, or tightening credit conditions. A macro allocation framework attempts to address this limitation by linking economic conditions to asset allocation decisions. The general idea is straightforward: if the economic environment can be classified into meaningful regimes, then portfolio weights can be adjusted to reflect which assets are most likely to perform well in each regime.

This project develops and tests a rule-based macro portfolio allocation model using monthly U.S. macroeconomic data and a diversified ETF universe. The first version of the model, referred to as V1, uses lagged macroeconomic indicators to classify the economy into discrete regimes such as Goldilocks, Reflation, Slowdown, and Stagflation. Each regime is then mapped to a fixed portfolio allocation across equities, Treasuries, credit, inflation-sensitive assets, and cash. The model is evaluated using a historical backtest and compared against standard benchmarks including SPY and a 60/40 portfolio.

The main objective of the project is not simply to maximize return, but to determine whether a macro regime framework can improve portfolio construction in a disciplined and economically interpretable way. In particular, the project examines whether this type of model can generate stronger risk-adjusted performance, reduce drawdowns, and respond more effectively to changing market environments than static allocations.

### 2. Research Question

This project asks whether lagged macroeconomic regime signals can improve portfolio allocation relative to static benchmark portfolios. More specifically, the analysis evaluates whether a discrete regime-based model can deliver either higher risk-adjusted returns, lower drawdowns, or both when compared with passive alternatives such as SPY and a traditional 60/40 allocation.

A second objective is to determine whether the model's value comes from genuine regime identification or simply from maintaining a more diversified and defensive asset mix. This distinction is important because a model that reduces volatility without adding predictive value is still useful, but it should be interpreted as a risk-management framework rather than an alpha-generating strategy.

## 3. Project Objective

The objective of Version 1 of the model is to build a transparent and reproducible allocation process that connects macroeconomic conditions to portfolio weights. The model is designed to be rule-based rather than discretionary, so that each allocation decision can be traced back to observable economic indicators. The broader goal is to test whether a simple macro framework can produce a portfolio that is more robust across changing market environments than static benchmark portfolios.

## 4. Structure of the Paper

The remainder of the paper is organized as follows. The next section describes the data and asset universe used in the model. The methodology section then explains how macroeconomic features were constructed, how regimes were defined, and how those regimes were translated into portfolio weights. The results section evaluates the strategy against benchmark portfolios using return, volatility, drawdown, turnover, and regime-level diagnostics. The paper concludes by assessing the strengths and weaknesses of the first version of the model.

## 5. Data

### 5.1 Macroeconomic Data

The macroeconomic dataset used in this model is constructed from publicly available series provided by the Federal Reserve Economic Data (FRED) database. All series are converted to a monthly frequency and aligned to month-end dates to ensure consistency with asset return data.

The model includes a broad set of indicators designed to capture inflation, economic growth, labor market conditions, monetary policy, the yield curve, and credit conditions. These dimensions are chosen because they are commonly associated with changes in asset class performance across economic cycles.

The specific macroeconomic series used are:

- **Inflation**
  - Headline CPI (CPIAUCSL)
  - Core CPI (CPILFESL)
- **Labor Market**
  - Unemployment Rate (UNRATE)
  - Nonfarm Payrolls (PAYEMS)
- **Economic Activity**
  - Industrial Production (INDPRO)
  - Housing Starts (HOUST)
  - Consumer Sentiment (UMCSENT)
- **Monetary Policy and Interest Rates**
  - Federal Funds Rate (FEDFUNDS)
  - 10-Year Treasury Yield (GS10)
  - 2-Year Treasury Yield (GS2)
  - 3-Month Treasury Bill Rate (TB3MS)
- **Credit Conditions**
  - BAA Corporate Bond Yield (BAA)
  - AAA Corporate Bond Yield (AAA)

From these raw series, additional features are constructed to better capture economic dynamics. These include year-over-year inflation rates, short-term annualized growth rates, multi-month changes in unemployment, yield curve spreads (such as the 10-year minus 3-month rate), and credit spreads between corporate and Treasury yields. Several of these features are smoothed using rolling averages to reduce short-term noise and avoid excessive sensitivity to monthly fluctuations.

All macroeconomic variables are lagged by one month when used in the model to ensure that portfolio decisions are based only on information that would have been available at the time.

---

### 5.2 Asset Universe

The portfolio is constructed using a diversified set of exchange-traded funds (ETFs), each representing a major asset class or risk factor. The goal is to ensure that the model can express different macroeconomic views through appropriate asset exposures.

The asset universe includes:

- **U.S. Equities**
  - SPY: Large-cap U.S. equities
  - QQQ: Growth-oriented equities with higher duration sensitivity
  - IWM: Small-cap equities with stronger sensitivity to domestic economic cycles
- **International Equities**
  - EFA: Developed markets ex-U.S.
  - EEM: Emerging markets
- **Fixed Income**
  - IEF: Intermediate-duration U.S. Treasuries
  - TLT: Long-duration U.S. Treasuries
- **Credit**
  - LQD: Investment-grade corporate bonds
  - HYG: High-yield corporate bonds
- **Inflation and Real Assets**
  - TIP: Treasury Inflation-Protected Securities
  - GLD: Gold
  - DBC: Broad commodity exposure
  - VNQ: Real estate investment trusts (REITs)
- **Cash**
  - BIL: Short-term Treasury bills

This asset set allows the model to allocate across growth-sensitive assets, defensive assets, inflation hedges, and credit instruments depending on the prevailing macroeconomic regime.

---

### 5.3 Sample Period and Frequency

All data is converted to a monthly frequency and aligned to the last calendar day of each month. The combined dataset begins in January 2004, which reflects the earliest period where a sufficient number of ETFs in the asset universe have available price history.

Because the model uses lagged macroeconomic signals to determine portfolio weights, the first usable return observation occurs one month after the initial data point. As a result, the effective backtest period begins in early 2004 and proceeds on a monthly basis through the end of the sample.

All returns are calculated using adjusted closing prices, ensuring that dividends and distributions are included in the total return series.

---

## 6. Model V1: Discrete Regime Allocation

### 6.1 Feature Engineering

The first step in constructing the model is to transform raw macroeconomic data into features that better capture changes in economic conditions. Rather than relying on levels alone, the model focuses on directional and rate-of-change measures, which are more informative for identifying turning points in the economic cycle.

For inflation, both headline and core CPI are converted into year-over-year growth rates, along with short-term annualized changes and multi-month differences. This allows the model to distinguish between stable inflation, accelerating inflation, and disinflationary environments.

For growth, several indicators are used in combination. These include changes in the unemployment rate, payroll growth, industrial production, housing activity, and consumer sentiment. Using multiple series helps reduce the risk that any single noisy indicator drives the classification.

Interest rate and policy conditions are captured using the federal funds rate, real interest rates, and the direction of policy changes over time. Yield curve information is incorporated through spreads such as the 10-year minus 3-month Treasury rate and the 10-year minus 2-year rate. These measures are commonly associated with forward-looking expectations about economic growth.

Credit conditions are measured using corporate bond spreads, including the difference between BAA and AAA yields and the spread between corporate yields and Treasury rates. These indicators reflect the degree of financial stress or ease in credit markets.

To reduce noise, several features are smoothed using short rolling averages. This prevents the model from reacting excessively to temporary fluctuations in monthly data. The result is a set of macro features that capture inflation trends, growth momentum, policy stance, yield curve shape, and credit conditions in a consistent and interpretable way.

---

### 6.2 Regime Classification

The model classifies the macroeconomic environment using a multi-step scoring system. Each major economic dimension is first translated into a numerical score based on a set of rules applied to the engineered features.

An **inflation score** reflects whether inflation is elevated, accelerating, or stable. A **growth score** captures the strength of economic activity using labor market data, production, and sentiment indicators. Additional scores are constructed for monetary policy, the yield curve, and credit conditions.

These scores are then grouped into categorical states. For example, inflation can be classified as low, moderate, or high, while growth can be classified as strong, moderate, or weak. Policy, curve, and credit conditions are similarly categorized into interpretable states such as restrictive versus accommodative or tight versus loose.

The model then defines a **primary regime** based on the interaction between growth and inflation. This creates economically intuitive categories such as:

- Goldilocks (strong growth, low inflation)
- Reflation (strong growth, rising inflation)
- Slowdown (weakening growth, contained inflation)
- Stagflation (weak growth, high inflation)

Additional intermediate states such as Recovery and LateCycle are included to capture transitions between these core regimes.

A second layer, referred to as a **risk overlay**, incorporates policy stance, yield curve shape, and credit conditions. This layer is used to distinguish between benign and stressed environments within the same inflation-growth backdrop. For example, a slowdown with supportive credit conditions is treated differently from a slowdown with tight credit and an inverted yield curve.

The final regime label combines the primary regime and the risk overlay. This results in a set of discrete macroeconomic regimes that reflect both the direction of the economy and the level of financial stress.

---

### 6.3 Portfolio Construction

Once a regime is identified, the model assigns a fixed set of portfolio weights corresponding to that regime. Each regime is mapped to a predefined allocation across equities, fixed income, credit, real assets, and cash.

Equity exposure is expressed through U.S. large-cap, growth, and small-cap equities, as well as international developed and emerging markets. Fixed income exposure includes both intermediate and long-duration Treasuries, allowing the model to differentiate between environments where duration risk is beneficial versus harmful.

Credit exposure is included through investment-grade and high-yield bonds, which are sensitive to changes in economic conditions and financial stress. Real assets such as gold, commodities, inflation-protected securities, and real estate are used to capture inflationary environments and diversification benefits. Cash exposure provides a defensive component during periods of elevated risk.

The model is **long-only**, fully invested, and does not use leverage or short positions. Portfolio weights are defined ex ante and are not optimized using historical returns. This ensures that the allocation framework remains transparent and avoids overfitting to past data.

Because not all ETFs are available over the full sample period, the model adjusts for missing assets by rescaling weights across the available assets in each month. This preserves the intended allocation structure without introducing artificial returns or dropping early observations.

---

### 6.4 Backtest Design

The backtest is conducted at a monthly frequency. At the end of each month, the model observes macroeconomic data and assigns a regime classification. To avoid look-ahead bias, all macro signals are lagged by one month. This means that portfolio weights for a given month are based only on information that would have been available at the end of the previous month.

Portfolio returns are calculated using total return data derived from adjusted ETF prices. The model is rebalanced monthly according to the regime-based weights.

Transaction costs are incorporated through a simple turnover-based model. Monthly turnover is calculated as half the sum of absolute changes in portfolio weights, and a fixed cost is applied per unit of turnover. This allows the analysis to compare both gross and net performance.

The strategy is evaluated against several benchmark portfolios, including a fully invested U.S. equity index (SPY), a traditional 60/40 stock-bond portfolio, and a diversified multi-asset benchmark. Performance is assessed using cumulative returns, volatility, Sharpe ratio, drawdowns, turnover, and regime-level diagnostics.

This framework provides a consistent and realistic evaluation of how a rule-based macro allocation model would have performed in real time.

## 7. V1 Results

### 7.1 Overall Performance

The first version of the macro allocation model produced a net compound annual growth rate of **8.67%** over the backtest sample. This was slightly below the gross return of **8.79%**, indicating that transaction costs had only a small effect on overall performance. For comparison, SPY generated a higher CAGR of **10.56%**, while the traditional 60/40 benchmark returned **8.00%** and the All Weather-style benchmark returned **6.70%**.

In risk-adjusted terms, the strategy compared more favorably. The net Sharpe ratio of the model was **0.88**, which was materially higher than SPY's Sharpe ratio of **0.73** and close to the **0.90** Sharpe ratio of the 60/40 benchmark. The All Weather-style benchmark posted a Sharpe ratio of **0.84**. This indicates that the model delivered a smoother return stream than a pure equity benchmark, although it did not generate a return premium relative to equities.

These results suggest that V1 functioned more as a risk-managed allocation strategy than as a high-alpha tactical model. Its main benefit was not higher absolute return, but improved return stability relative to a fully invested equity benchmark.

---

### 7.2 Risk and Drawdowns

A major strength of the model was downside control. The maximum drawdown of the net strategy was **-18.46%**, which was dramatically smaller than the **-50.78%** maximum drawdown experienced by SPY. The drawdown was also smaller than the **-29.46%** drawdown of the 60/40 benchmark and modestly better than the **-21.24%** drawdown of the All Weather-style benchmark.

Annualized volatility was **9.85%** for the net strategy, compared with **14.50%** for SPY, **8.89%** for the 60/40 benchmark, and **7.93%** for the All Weather-style portfolio. This places the model between a traditional stock-bond portfolio and an equity-only benchmark in terms of risk.

Taken together, the volatility and drawdown results indicate that the model was successful at reducing portfolio stress during adverse market environments. However, that risk reduction came at the cost of lower upside participation in strong equity markets.

---

### 7.3 Subperiod Performance

Subperiod results help clarify when the model worked well and when it struggled. From **2004 to 2007**, the strategy generated a net CAGR of **10.88%**, outperforming SPY's **8.50%**. From **2008 to 2012**, the strategy again outperformed, returning **6.15%** annually compared with **2.94%** for SPY.

The pattern reversed in later years. From **2013 to 2019**, the strategy returned **9.12%**, while SPY returned **13.77%**. From **2020 onward**, the strategy returned **8.44%** versus **14.68%** for SPY. In both later subperiods, the strategy still remained close to or slightly below the 60/40 benchmark, but it clearly lagged a strong equity market.

This breakdown shows that the model performed better in environments characterized by crisis, recovery, or elevated macro uncertainty, but had difficulty keeping pace during prolonged bull markets. That pattern is consistent with a framework that is better at risk reduction than at maximizing upside capture.

---

### 7.4 Regime-Level Performance

The regime diagnostics provide additional evidence that the model's main contribution came from defense rather than prediction. Across most regimes, the strategy produced **negative average alpha relative to SPY**. For example, average alpha versus SPY was negative in Goldilocks, Reflation, Recovery, Slowdown, Overheat, LateCycle, and SoftLanding environments. The only regime with clearly positive alpha relative to SPY was **Stagflation**, where the average monthly alpha was approximately **2.64%**. However, this result was based on only **four observations**, so it should not be treated as strong evidence of a robust edge.

Hit rates versus SPY also support this conclusion. In several regimes, the model beat SPY less than half the time. Goldilocks, Recovery, Slowdown, Overheat, LateCycle, and SoftLanding all had hit rates below 50% against SPY. Reflation and Transition were closer to even, while Stagflation again appeared favorable but on a very small sample.

These results indicate that the regime classification did not consistently produce superior equity timing. Instead, the model often traded some upside for lower volatility and more diversification. That distinction is important because it suggests that the V1 framework should be interpreted primarily as a disciplined allocation system rather than as a reliable source of tactical alpha.

---

Table 1: V1 Performance Summary

| Series | Months | CAGR | Vol | Sharpe | Sortino | Calmar | MaxDD |
|---|---|---|---|---|---|---|---|
| Strategy_Gross | 266 | 0.088 | 0.098 | 0.892 | 1.572 | 0.477 | -0.184 |
| Strategy_Net | 266 | 0.087 | 0.099 | 0.880 | 1.547 | 0.470 | -0.185 |
| SPY | 266 | 0.106 | 0.145 | 0.729 | 1.229 | 0.208 | -0.508 |
| 60_40 | 266 | 0.080 | 0.089 | 0.900 | 1.532 | 0.271 | -0.295 |
| AllWeather | 266 | 0.067 | 0.079 | 0.844 | 1.421 | 0.315 | -0.212 |

### 7.5 Turnover and Implementation

Implementation frictions were not a major weakness of the model. Average monthly turnover was **0.0900**, or about **9.0%** of the portfolio. Median monthly turnover was **0**, which indicates that many months involved no allocation changes at all. Maximum monthly turnover reached **0.49**, reflecting a small number of larger reallocation events during regime shifts.

The regime switch rate was **35.34%**, meaning that the model changed its regime classification in a little over one-third of months. Despite this, the average transaction cost was only **0.00008997** per month under the assumed cost model. This confirms that the gap between gross and net returns was small and that implementation costs did not materially alter the main conclusions.

In other words, the model was not failing because it traded too frequently. The larger issue was that its regime-based allocations did not generate enough incremental return to consistently overcome the performance of a simple equity benchmark.

---

### 7.6 Return Decomposition

The sleeve contribution analysis shows that the largest share of return came from **U.S. equities**, which contributed an average of approximately **0.00405** per month. The next-largest contributors were **real assets** at roughly **0.00130** and **international equities** at roughly **0.00114**. Credit contributed **0.00058**, Treasuries contributed **0.00029**, and cash contributed only **0.00009**.

This is an important result because it shows that the strategy's performance was still driven primarily by equity exposure. Defensive sleeves such as Treasuries and cash helped stabilize the portfolio, but they were not the dominant source of return. Real assets provided a meaningful secondary contribution, which is consistent with the model's use of inflation-sensitive allocations in certain regimes.

Overall, the decomposition reinforces the earlier conclusion that the model behaved like a diversified, risk-managed portfolio with a macro overlay rather than a strongly predictive tactical strategy. Its strength was in portfolio balance and downside moderation, not in repeatedly identifying the highest-return asset mix ahead of time.

---

### 7.7 Summary of V1 Results

The first version of the model produced a credible and internally consistent set of results. It achieved lower volatility and substantially lower drawdowns than SPY, while generating returns that were broadly competitive with a traditional 60/40 portfolio. At the same time, it did not demonstrate consistent alpha versus a passive equity benchmark, particularly during long bull market periods.

The evidence therefore suggests that V1 succeeded as a macro-aware risk allocation system, but not as a strong return-maximizing model. This distinction motivates the next section, which examines the main limitations of the V1 design and explains why a second version of the model is needed.

## 8. Limitations of V1

While the first version of the model produces stable and interpretable results, several limitations emerge from both the backtest and the diagnostic analysis. These limitations help explain why the model improves risk characteristics but does not consistently generate excess returns relative to an equity benchmark.

---

### 8.1 Lack of Persistent Alpha

The most direct limitation is the absence of consistent alpha relative to SPY. Although the model delivers a higher Sharpe ratio and significantly lower drawdowns, it underperforms SPY in terms of absolute return. This is reinforced by the regime-level diagnostics, where average alpha versus SPY is negative in most regimes.

This suggests that the model is not reliably identifying environments in which it should increase or decrease equity exposure. Instead, it is frequently positioned more defensively than necessary during strong market periods, leading to missed upside.

---

### 8.2 Underperformance in Bull Markets

Subperiod analysis shows a clear pattern: the model performs well during periods of economic stress or transition, but lags significantly during extended bull markets. From 2013 onward, when equity markets experienced sustained growth, the strategy consistently underperformed SPY.

This behavior is consistent with a model that reduces risk effectively but fails to re-risk aggressively enough when conditions are favorable. As a result, the model sacrifices too much upside in environments where equities dominate portfolio returns.

---

### 8.3 Weak Regime Discrimination

The regime classification framework does not produce strong separation between favorable and unfavorable environments. Hit rates versus SPY are below or near 50 percent in many regimes, indicating that the model is not consistently identifying when its allocation should outperform.

Additionally, the only regime with clearly positive alpha relative to SPY is Stagflation, but this result is based on a very small number of observations. This raises concerns about statistical reliability and suggests that the model's apparent success in that regime may not generalize.

---

### 8.4 Discrete Regime Structure

The model relies on a discrete regime framework in which each economic environment is mapped to a fixed set of portfolio weights. While this structure is intuitive and easy to interpret, it introduces rigidity.

Small changes in macro signals can cause abrupt shifts in regime classification, even when the underlying economic environment has not meaningfully changed. At the same time, two observations within the same regime receive identical portfolio weights, even if their underlying macro conditions differ in magnitude or direction.

This loss of granularity limits the model's ability to adjust exposures smoothly and may reduce its responsiveness to evolving conditions.

---

### 8.5 Lagging Indicators

All macroeconomic inputs are lagged by one month to avoid look-ahead bias. While this is necessary for a realistic backtest, it introduces a structural delay in the model's response to changing conditions.

Many macro indicators, such as unemployment and inflation, are inherently backward-looking. As a result, the model often reacts after turning points have already begun, particularly during rapid market transitions. This delay can lead to suboptimal positioning during both recoveries and downturns.

---

### 8.6 Static Weight Mapping

Each regime is associated with a fixed set of portfolio weights that do not adapt over time. These weights are not estimated or optimized using data, and they do not adjust based on the strength of signals within a regime.

While this design avoids overfitting, it also limits flexibility. For example, a moderate inflation increase and a sharp inflation surge may both map to the same regime, even though they may warrant different portfolio responses. The static mapping therefore reduces the model's ability to scale exposures appropriately.

---

### 8.7 Interpretation of the Model

Taken together, these limitations suggest that the V1 model should not be viewed as a strong predictive or alpha-generating system. Instead, it behaves more like a structured multi-asset allocation framework with a macro overlay.

Its primary strength lies in reducing volatility and drawdowns while maintaining reasonable return levels. However, it does not consistently identify opportunities to outperform equity markets, particularly in strong growth environments.

---

### 8.8 Implications for Model Development

These findings motivate the need for a second version of the model. In particular, improvements should focus on:

- replacing discrete regime buckets with a continuous scoring framework
- allowing portfolio weights to adjust dynamically with signal strength
- improving responsiveness to changing macro conditions
- preserving diversification benefits while reducing unnecessary defensiveness

The next section outlines a revised model design that attempts to address these limitations.

## 9. Model V2: Continuous Macro Allocation Framework

### 9.1 Motivation for V2

The results from V1 indicate that the model is effective at reducing volatility and drawdowns, but does not generate consistent alpha relative to equity benchmarks. The primary issue is not the use of macro data itself, but how that information is translated into portfolio decisions.

In particular, the discrete regime structure forces the model into rigid allocations that do not adjust smoothly as conditions change. This leads to two key problems. First, the model often remains too defensive during

strong equity environments. Second, it does not scale exposures appropriately when macro signals strengthen or weaken within the same regime.

To address these issues, V2 replaces the discrete regime framework with a continuous scoring system that directly maps macro signals to portfolio weights.

---

### 9.2 Continuous Macro Scoring

Instead of assigning each observation to a single regime, V2 constructs a continuous macro score using the same underlying components:

- inflation conditions
- growth conditions
- monetary policy stance
- yield curve shape
- credit conditions

Each component is normalized so that it can be compared across time. These normalized signals are then combined into a composite macro score representing the overall economic environment.

Rather than classifying the economy into categories such as Goldilocks or Stagflation, the model treats macro conditions as existing along a spectrum. This allows for more gradual transitions and avoids abrupt allocation changes caused by small shifts in input variables.

---

### 9.3 Dynamic Portfolio Mapping

In V2, portfolio weights are no longer fixed by regime. Instead, they are functions of the continuous macro score.

The core idea is to map macro conditions to asset exposures in a smooth and monotonic way. For example:

- stronger growth signals increase equity exposure
- weaker growth signals increase defensive assets such as Treasuries and cash
- higher inflation signals increase exposure to real assets such as commodities and gold
- tighter credit conditions reduce exposure to high-yield and cyclical assets

This mapping can be implemented using linear scaling, piecewise functions, or bounded transformations to ensure weights remain within reasonable limits.

As a result, portfolio weights evolve continuously over time rather than shifting abruptly between predefined allocations.

---

### 9.4 Improved Responsiveness

Because the model operates on continuous signals, it can respond more quickly to changes in macro conditions. Even small shifts in growth or inflation indicators will gradually adjust portfolio weights, rather than waiting for a full regime transition.

This reduces the lag inherent in V1 and allows the model to participate more effectively in early-stage recoveries and late-cycle expansions.

---

### 9.5 Expected Improvements

The V2 framework is designed to address the key weaknesses identified in V1:

- **Reduced rigidity**: Continuous mapping replaces discrete regime jumps
- **Better upside capture**: Equity exposure can scale higher in strong environments
- **More precise positioning**: Allocation reflects signal strength, not just direction
- **Improved adaptability**: The model adjusts incrementally as conditions evolve

At the same time, the model retains the core advantages of the V1 framework, including transparency, economic interpretability, and diversification across asset classes.

---

### 9.6 Implementation Plan

The next step is to implement the continuous scoring system and weight mapping in code, replacing the regime-based allocation logic from V1. The same backtesting framework will be used to evaluate performance, allowing for a direct comparison between the two approaches.

The results of V2 will determine whether a continuous macro framework can improve both absolute returns and risk-adjusted performance relative to the original model.

## 10. Results: Continuous Macro–Momentum Allocation Model (V2)

The second version of the model replaces the discrete regime classification used in V1 with a continuous scoring framework. Instead of assigning each period to a single macroeconomic regime, V2 constructs a composite signal from inflation, growth, policy, and momentum indicators. This signal is then mapped to portfolio weights using a nonlinear allocation function.

The goal of this modification was to increase flexibility and allow the portfolio to respond more gradually to changes in economic conditions. In theory, a continuous signal should reduce abrupt allocation shifts and better capture transitions between regimes.

### 10.1 Overall Performance

Over the full sample period, the V2 strategy produced a compound annual growth rate of approximately 4.9% before transaction costs and 4.8% after costs. This is materially lower than the return of the S&P 500 over the same period, which exceeded 10% annually.

Volatility for the strategy was significantly lower, at roughly 7.5% per year compared to nearly 15.8% for the S&P 500. As a result, the strategy achieved a Sharpe ratio of approximately 0.65, which is close to but still below that of the S&P 500 and meaningfully below that of a traditional 60/40 portfolio.

While the model successfully reduced volatility, it did not convert this reduction into improved risk-adjusted performance. The return sacrifice was too large relative to the volatility reduction.

### 10.2 Drawdown and Downside Risk

The maximum drawdown of the V2 strategy was approximately -23%, compared to a drawdown of roughly -46% for the S&P 500. This confirms that the model does provide some downside protection during adverse market environments.

Table 2: V2 Performance Summary

| Series | Months | CAGR | Vol | Sharpe | Sortino | Calmar | MaxDD |
|---|---|---|---|---|---|---|---|
| V2_Strategy_Gross | 213 | 0.049 | 0.075 | 0.650 | 1.023 | 0.212 | -0.230 |
| V2_Strategy_Net | 213 | 0.048 | 0.075 | 0.640 | 1.007 | 0.209 | -0.231 |
| SPY | 213 | 0.110 | 0.158 | 0.695 | 0.956 | 0.237 | -0.463 |
| 60_40 | 213 | 0.082 | 0.101 | 0.815 | 1.048 | 0.308 | -0.266 |
| AllWeather | 213 | 0.063 | 0.083 | 0.758 | 1.013 | 0.316 | -0.199 |

However, the magnitude of drawdown reduction is not sufficient to offset the loss in return. In addition, the strategy still experienced meaningful declines, indicating that the defensive positioning was not aggressive enough during periods of market stress.

The Sortino ratio, which focuses on downside volatility, was slightly above 1.0. This is an improvement relative to the S&P 500 but remains comparable to simpler diversified portfolios such as the All Weather portfolio.

### 10.3 Comparison to Benchmark Portfolios

When compared to standard benchmarks, the limitations of the V2 model become more apparent.

- The S&P 500 delivered substantially higher returns with only moderately higher risk.
- A 60/40 portfolio achieved a higher Sharpe ratio and better overall efficiency.
- The All Weather portfolio produced a similar return profile but with better drawdown characteristics.

In effect, the V2 strategy behaves like a conservative, low-volatility allocation without offering a clear advantage over existing portfolio constructions. It neither outperforms equities in strong markets nor provides sufficiently strong protection in weak markets.

### 10.4 Interpretation of Results

The results suggest that the continuous scoring approach reduces the effectiveness of the allocation signal. By averaging a large number of macroeconomic indicators, the model produces a signal that is smooth but lacks conviction. As a result, portfolio weights tend to remain in a moderate range rather than shifting decisively between risk-on and risk-off positions.

Although momentum was included in the model, it was combined with slower-moving macroeconomic variables. This limits its ability to act as a timely indicator of market conditions. The allocation process therefore remains dominated by lagging information.

Additionally, diversification across multiple asset sleeves further dampens performance. Even in favorable environments, capital is spread across lower-return assets, which reduces upside participation.

### 10.5 Key Takeaways

The results suggest that the continuous scoring approach reduces the effectiveness of the allocation signal. By averaging a large number of macroeconomic indicators, the model produces a signal that is smooth but lacks conviction. As a result, portfolio weights tend to remain in a moderate range rather than shifting decisively between risk-on and risk-off positions.

Although momentum was included in the model, it was combined with slower-moving macroeconomic variables. This limits its ability to act as a timely indicator of market conditions. The allocation process therefore remains dominated by lagging information.

Additionally, diversification across multiple asset sleeves further dampens performance. Even in favorable environments, capital is spread across lower-return assets, which reduces upside participation.

## 11. Model Comparison: Discrete Regimes vs Continuous Scoring

This section compares the two portfolio construction approaches developed in this project. V1 uses a discrete regime classification framework, while V2 relies on a continuous composite scoring system. Both models are built on the same underlying macroeconomic data, but differ in how signals are translated into portfolio allocations.

The purpose of this comparison is to evaluate whether increasing model flexibility through continuous scoring leads to improved performance.

### 11.1 Structural Differences

The primary distinction between the two models lies in how macroeconomic information is processed.

- V1 (Discrete Regimes) assigns each time period to a specific macro regime based on threshold conditions. Portfolio allocations are tied directly to these regimes, resulting in clear and distinct positioning.
- V2 (Continuous Scoring) aggregates macro and momentum indicators into a blended score, which is then mapped smoothly into portfolio weights. Allocations adjust incrementally rather than switching between predefined states.

This difference creates a tradeoff between conviction and smoothness. V1 produces sharper allocation shifts, while V2 emphasizes gradual transitions.

### 11.2 Performance Comparison

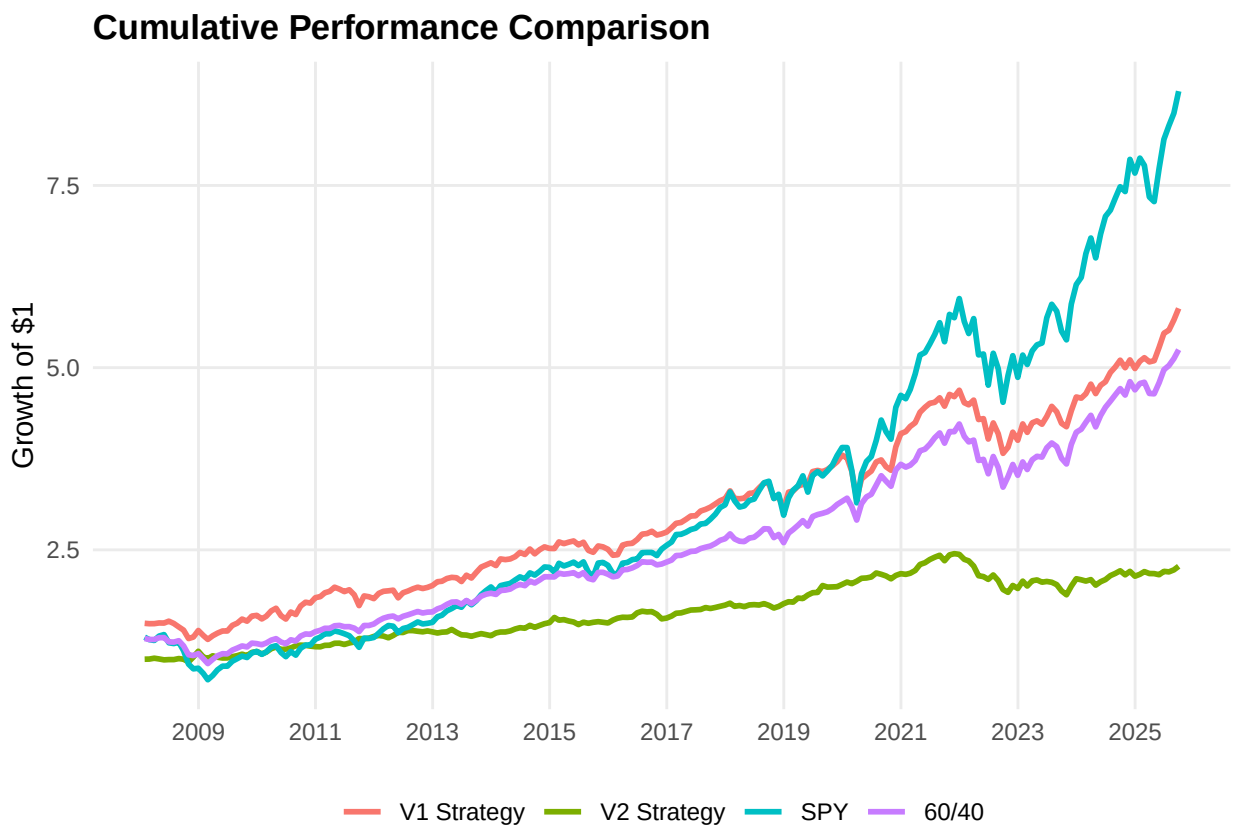
Across the full sample, V1 outperforms V2 on both return and risk-adjusted metrics.

V1 achieves higher returns while maintaining similar or slightly higher volatility.

V2 produces lower volatility, but the reduction is not sufficient to compensate for its weaker returns.

Risk-adjusted measures such as Sharpe and Sortino ratios favor V1 or remain comparable to simple benchmark portfolios.

The key observation is that V2 does not improve upon V1 despite being a more flexible and complex model.




### 11.3 Allocation Behavior

The difference in performance can be traced to how each model allocates capital.

V1's regime-based structure leads to more decisive positioning. During favorable economic conditions, the model meaningfully increases exposure to risk assets. In adverse environments, it shifts more aggressively toward defensive assets.

In contrast, V2 tends to remain in a moderate allocation range. Because the signal is constructed as an average of multiple inputs, extreme values are rare. This results in limited exposure to high-conviction opportunities and insufficient defensive positioning during downturns.

The outcome is a portfolio that is consistently diversified but rarely optimized for prevailing conditions.

### 11.4 Role of Momentum

Momentum was introduced in V2 to address the lagging nature of macroeconomic indicators. However, its impact is limited by the structure of the model.

Rather than driving allocation decisions directly, momentum is blended with macro signals. This reduces its influence and prevents it from acting as a leading indicator. As a result, the model does not fully capture market trends, particularly during sustained equity rallies.

This highlights a structural issue: combining fast-moving and slow-moving signals through simple averaging can dilute the effectiveness of both.

### 11.5 Complexity vs Effectiveness

V2 is more complex than V1 in several dimensions:

- More input features
- Continuous transformations
- Nonlinear allocation mapping
- Additional constraints and overrides

Despite this added complexity, the model does not deliver improved results. In fact, performance deteriorates relative to the simpler framework.

This suggests that the limiting factor is not the sophistication of the allocation function, but the quality and structure of the underlying signal.

### 11.6 Key Takeaways

The comparison between V1 and V2 leads to several conclusions:

- Discrete regime models can outperform more flexible continuous models when signals are weak or noisy.
- Increasing smoothness in allocation reduces both volatility and return, often without improving risk-adjusted performance.
- Combining macro and momentum signals through averaging reduces the effectiveness of both.
- Additional model complexity does not guarantee better outcomes.

Overall, V1 provides a more effective framework for translating macroeconomic information into portfolio decisions. V2 demonstrates that greater flexibility can come at the cost of reduced conviction and weaker performance.

## 12. Conclusion

This project evaluated whether macroeconomic information can be used to construct a dynamic asset allocation strategy that improves risk-adjusted returns relative to traditional benchmarks.

Two models were developed. The first used a discrete regime classification framework, while the second introduced a continuous scoring system that combined macroeconomic indicators with momentum.

### 12.1 Summary of Findings

The results show that both models are capable of reducing portfolio volatility and drawdowns relative to an equity-only benchmark. However, neither model is able to generate excess returns sufficient to outperform standard portfolios such as the S&P 500 or a 60/40 allocation.

The discrete regime model (V1) performed better than the continuous scoring model (V2). V1 produced higher returns and more effective allocation shifts, while V2 generated smoother but less impactful positioning. The continuous framework reduced volatility but also significantly reduced return, leading to weaker overall performance.

### 12.2 Interpretation

The key limitation across both models is the predictive power of the underlying signals. The macroeconomic variables used in this analysis are largely slow-moving and often lag market behavior. As a result, the models are not able to anticipate major market turning points or capture sustained trends early enough.

In V2, this limitation is amplified by averaging multiple signals together. While this approach stabilizes the signal, it also reduces its magnitude and dampens its ability to drive meaningful allocation changes. The inclusion of momentum partially addresses this issue, but its influence is constrained by the model structure.

Overall, the results suggest that macroeconomic averaging alone is insufficient to produce a strong tactical allocation signal.

### 12.3 Implications for Portfolio Construction

From a practical perspective, the models developed in this project behave more like risk management overlays than return-enhancing strategies.

They are effective at:

- Reducing volatility
- Limiting drawdowns
- Maintaining diversified exposure

However, they are not effective at:

- Generating alpha
- Timing market cycles with precision
- Outperforming simple benchmark portfolios

This implies that macro-based allocation models may be better suited as a component within a broader investment framework rather than as a standalone strategy.

### 12.4 Limitations

Several limitations should be acknowledged.

- The model relies on a fixed set of macroeconomic indicators, which may not fully capture the drivers of asset returns.
- Signal construction is based on linear transformations and averaging, which may oversimplify complex relationships.
- The backtest assumes stable relationships between macro variables and asset performance over time.
- Transaction costs and implementation frictions, while included, are simplified.

These factors may impact the generalizability of the results.

### 12.5 Future Directions

The findings suggest that future improvements should focus on signal design rather than additional complexity in the allocation process.

Potential directions include:

- Prioritizing momentum or trend-following signals as primary drivers
- Using macroeconomic variables as filters rather than core signals
- Reducing the number of inputs to focus on higher-quality indicators
- Exploring alternative modeling approaches that better capture nonlinear relationships

A shift toward more responsive and forward-looking signals is likely necessary to improve performance.

### 12.6 Final Assessment

The models developed in this project demonstrate that it is possible to construct a systematic macro-driven allocation strategy that reduces risk. However, they also show that reducing risk alone is not sufficient to outperform established benchmarks.

The primary challenge is not implementation, but signal effectiveness. Without a strong predictive signal, increased model complexity and flexibility do not translate into better outcomes.

As a result, the evidence from this analysis does not support the use of averaged macroeconomic indicators as a standalone framework for tactical asset allocation.

## Appendix A: V1 Code

```r
# ==========================================================
# Step 1: Pull and prepare macro data in monthly format
# Improved version:
# - more indicators
# - explicit month-end alignment
# - smoother / less noisy features
# - better growth / inflation / policy / credit coverage
# ==========================================================

suppressPackageStartupMessages({
  library(tidyverse)
  library(lubridate)
  library(readr)
  library(zoo)
})

Sys.setlocale("LC_TIME", "C")

# ------------------------------
# Helper: load FRED fredgraph.csv
# Handles Excel serial dates or date strings
# Forces month-end alignment
# ------------------------------
fred_load <- function(path_or_url, value_name = "value") {
  x <- read_csv(path_or_url, show_col_types = FALSE)

  names(x)[1:2] <- c("date_raw", value_name)

  x %>%
    mutate(
      date = case_when(
        is.numeric(date_raw) ~ as.Date(date_raw, origin = "1899-12-30"),
        TRUE ~ suppressWarnings(as.Date(date_raw))
      )
    ) %>%
    filter(!is.na(date)) %>%
    transmute(
      date = ceiling_date(date, "month") - days(1),
      !!value_name := as.numeric(.data[[value_name]])
    ) %>%
    distinct(date, .keep_all = TRUE) %>%
    arrange(date)
}

# ------------------------------
# FRED series URLs
# Core inflation / labor / growth / policy / curve / credit
# ------------------------------
fred_urls <- list(
  CPIAUCSL   = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=CPIAUCSL",     # headline CPI
  CPILFESL   = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=CPILFESL",     # core CPI
  UNRATE     = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=UNRATE",       # unemployment rate
```

```r
  FEDFUNDS   = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=FEDFUNDS",     # fed funds
  GS10       = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=GS10",         # 10y Treasury
  GS2        = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=GS2",          # 2y Treasury
  TB3MS      = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=TB3MS",        # 3m T-bill
  PAYEMS     = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=PAYEMS",       # payrolls
  INDPRO     = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=INDPRO",       # industrial producti
  HOUST      = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=HOUST",        # housing starts
  UMCSENT    = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=UMCSENT",      # consumer sentiment
  BAA        = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=BAA",          # BAA corporate yield
  AAA        = "https://fred.stlouisfed.org/graph/fredgraph.csv?id=AAA"           # AAA corporate yield
)

# ----------------------------
# Load each macro series
# ----------------------------
cpi_headline <- fred_load(fred_urls$CPIAUCSL, "cpi")
cpi_core     <- fred_load(fred_urls$CPILFESL, "core_cpi")
unrate       <- fred_load(fred_urls$UNRATE,   "unrate")
fedfunds     <- fred_load(fred_urls$FEDFUNDS, "fedfunds")
gs10         <- fred_load(fred_urls$GS10,     "gs10")
gs2          <- fred_load(fred_urls$GS2,      "gs2")
tb3ms        <- fred_load(fred_urls$TB3MS,    "tb3ms")
payems       <- fred_load(fred_urls$PAYEMS,   "payems")
indpro       <- fred_load(fred_urls$INDPRO,   "indpro")
houst        <- fred_load(fred_urls$HOUST,    "houst")
umcsent      <- fred_load(fred_urls$UMCSENT,  "umcsent")
baa          <- fred_load(fred_urls$BAA,      "baa")
aaa          <- fred_load(fred_urls$AAA,      "aaa")

# ----------------------------
# Merge into one monthly macro table
# ----------------------------
macro_raw <- cpi_headline %>%
  full_join(cpi_core, by = "date") %>%
  full_join(unrate,   by = "date") %>%
  full_join(fedfunds, by = "date") %>%
  full_join(gs10,     by = "date") %>%
  full_join(gs2,      by = "date") %>%
  full_join(tb3ms,    by = "date") %>%
  full_join(payems,   by = "date") %>%
  full_join(indpro,   by = "date") %>%
  full_join(houst,    by = "date") %>%
  full_join(umcsent,  by = "date") %>%
  full_join(baa,      by = "date") %>%
  full_join(aaa,      by = "date") %>%
  arrange(date)

# ----------------------------
# Helper: annualized multi-month growth
# Example: 3-month annualized inflation / growth
# ----------------------------
ann_growth <- function(x, k) {
  100 * ((x / lag(x, k))^(12 / k) - 1)
```

```r
}

# ----------------------------
# Build derived macro features
# Notes:
# - use yoy for medium-term trend
# - use 3m annualized rates for turning points
# - use 3m rolling averages on noisy deltas
# - create interpretable policy / curve / credit features
# ----------------------------
macro_monthly <- macro_raw %>%
  filter(date >= as.Date("1995-01-31")) %>%
  mutate(
    # Inflation levels / trends
    cpi_yoy               = 100 * (cpi / lag(cpi, 12) - 1),
    core_cpi_yoy          = 100 * (core_cpi / lag(core_cpi, 12) - 1),
    cpi_3m_ann            = ann_growth(cpi, 3),
    core_cpi_3m_ann       = ann_growth(core_cpi, 3),
    cpi_yoy_6m_delta      = cpi_yoy - lag(cpi_yoy, 6),
    core_cpi_yoy_6m_delta = core_cpi_yoy - lag(core_cpi_yoy, 6),

    # Labor / growth
    unrate_3m_avg         = rollmean(unrate, 3, fill = NA, align = "right"),
    unrate_6m_delta       = unrate - lag(unrate, 6),
    payems_yoy            = 100 * (payems / lag(payems, 12) - 1),
    payems_3m_ann         = ann_growth(payems, 3),
    indpro_yoy            = 100 * (indpro / lag(indpro, 12) - 1),
    indpro_3m_ann         = ann_growth(indpro, 3),
    houst_yoy             = 100 * (houst / lag(houst, 12) - 1),
    umcsent_6m_delta      = umcsent - lag(umcsent, 6),

    # Policy / real rates
    real_fedfunds         = fedfunds - cpi_yoy,
    real_10y              = gs10 - cpi_yoy,

    # Curve / rates
    term_spread_10y3m     = gs10 - tb3ms,
    term_spread_10y2y     = gs10 - gs2,
    front_end_spread_2y3m = gs2 - tb3ms,

    # Credit
    baa_aaa_spread        = baa - aaa,
    baa_10y_spread        = baa - gs10
  ) %>%
  mutate(
    # -------------------------
    # Noise reduction
    # Smooth noisy directional signals
    # -------------------------
    cpi_yoy_6m_delta_smooth      = rollmean(cpi_yoy_6m_delta, 3, fill = NA, align = "right"),
    core_cpi_yoy_6m_delta_smooth = rollmean(core_cpi_yoy_6m_delta, 3, fill = NA, align = "right"),
    unrate_6m_delta_smooth       = rollmean(unrate_6m_delta, 3, fill = NA, align = "right"),
    payems_3m_ann_smooth         = rollmean(payems_3m_ann, 3, fill = NA, align = "right"),
```

```r
    indpro_3m_ann_smooth         = rollmean(indpro_3m_ann, 3, fill = NA, align = "right"),
    houst_yoy_smooth             = rollmean(houst_yoy, 3, fill = NA, align = "right"),
    baa_10y_spread_smooth        = rollmean(baa_10y_spread, 3, fill = NA, align = "right"),

    # -------------------------
    # Binary directional flags
    # Useful for later regime construction
    # -------------------------
    inflation_trend_up = if_else(cpi_yoy_6m_delta_smooth > 0, 1, 0, missing = NA_integer_),
    labor_trend_up     = if_else(unrate_6m_delta_smooth < 0, 1, 0, missing = NA_integer_),
    growth_trend_up    = if_else(
      payems_3m_ann_smooth > 0 &
      indpro_3m_ann_smooth > 0,
      1, 0, missing = NA_integer_
    ),
    curve_positive     = if_else(term_spread_10y3m > 0, 1, 0, missing = NA_integer_),
    credit_loose       = if_else(baa_10y_spread_smooth < lag(baa_10y_spread_smooth, 6), 1, 0, missing =
  ) %>%
  select(
    date,

    # Raw levels
    cpi, core_cpi, unrate, fedfunds, gs10, gs2, tb3ms,
    payems, indpro, houst, umcsent, baa, aaa,

    # Inflation
    cpi_yoy, core_cpi_yoy, cpi_3m_ann, core_cpi_3m_ann,
    cpi_yoy_6m_delta, core_cpi_yoy_6m_delta,
    cpi_yoy_6m_delta_smooth, core_cpi_yoy_6m_delta_smooth,

    # Growth / labor
    unrate_3m_avg, unrate_6m_delta, unrate_6m_delta_smooth,
    payems_yoy, payems_3m_ann, payems_3m_ann_smooth,
    indpro_yoy, indpro_3m_ann, indpro_3m_ann_smooth,
    houst_yoy, houst_yoy_smooth,
    umcsent_6m_delta,

    # Policy / curve / credit
    real_fedfunds, real_10y,
    term_spread_10y3m, term_spread_10y2y, front_end_spread_2y3m,
    baa_aaa_spread, baa_10y_spread, baa_10y_spread_smooth,

    # Flags
    inflation_trend_up, labor_trend_up, growth_trend_up,
    curve_positive, credit_loose
  ) %>%
  arrange(date)

# ----------------------------
# Inspect result
# ----------------------------
glimpse(macro_monthly)
print(head(macro_monthly, 15))
```

```r
print(tail(macro_monthly, 15))

# ----------------------------
# Optional: save intermediate file
# ----------------------------
write_csv(macro_monthly, "macro_monthly_step1_improved.csv")



# ==========================================================
# Step 2: Convert macro features into regime signals
# ==========================================================

# ==========================================================
# Step 2: Convert macro features into regime signals
# Improved version:
# - score-based regime classification
# - confirmation from growth, policy, curve, and credit
# - less sensitive to one noisy series
# ==========================================================

macro_regimes <- macro_monthly %>%
  mutate(
    # ---------------------------------------------------
    # 1. Inflation score
    # Higher score = inflation running hotter / worsening
    # ---------------------------------------------------
    inflation_score =
      1 * if_else(cpi_yoy > 3, 1, 0, missing = NA_real_) +
      1 * if_else(core_cpi_yoy > 3, 1, 0, missing = NA_real_) +
      1 * if_else(cpi_3m_ann > cpi_yoy, 1, 0, missing = NA_real_) +
      1 * if_else(cpi_yoy_6m_delta_smooth > 0, 1, 0, missing = NA_real_),

    # ---------------------------------------------------
    # 2. Growth score
    # Higher score = stronger cyclical growth backdrop
    # ---------------------------------------------------
    growth_score =
      1 * if_else(unrate_6m_delta_smooth < 0, 1, 0, missing = NA_real_) +
      1 * if_else(payems_3m_ann_smooth > 0, 1, 0, missing = NA_real_) +
      1 * if_else(indpro_3m_ann_smooth > 0, 1, 0, missing = NA_real_) +
      1 * if_else(houst_yoy_smooth > 0, 1, 0, missing = NA_real_) +
      1 * if_else(umcsent_6m_delta > 0, 1, 0, missing = NA_real_),

    # ---------------------------------------------------
    # 3. Policy score
    # Higher score = more restrictive monetary backdrop
    # ---------------------------------------------------
    policy_score =
      1 * if_else(real_fedfunds > 0, 1, 0, missing = NA_real_) +
      1 * if_else(fedfunds > lag(fedfunds, 6), 1, 0, missing = NA_real_) +
      1 * if_else(real_10y > 1, 1, 0, missing = NA_real_),
```

```r
# ---------------------------------------------------
# 4. Curve score
# Higher score = healthier term structure
# ---------------------------------------------------
curve_score =
  1 * if_else(term_spread_10y3m > 0, 1, 0, missing = NA_real_) +
  1 * if_else(term_spread_10y2y > 0, 1, 0, missing = NA_real_) +
  1 * if_else(front_end_spread_2y3m > 0, 1, 0, missing = NA_real_),

# ---------------------------------------------------
# 5. Credit score
# Higher score = easier / less stressed credit backdrop
# ---------------------------------------------------
credit_score =
  1 * if_else(baa_aaa_spread < lag(baa_aaa_spread, 6), 1, 0, missing = NA_real_) +
  1 * if_else(baa_10y_spread_smooth < lag(baa_10y_spread_smooth, 6), 1, 0, missing = NA_real_) +
  1 * if_else(baa_10y_spread < 3, 1, 0, missing = NA_real_),

# ---------------------------------------------------
# 6. Binary state buckets from the scores
# ---------------------------------------------------
inflation_state = case_when(
  inflation_score >= 3 ~ "High",
  inflation_score <= 1 ~ "Low",
  TRUE ~ "Moderate"
),

growth_state = case_when(
  growth_score >= 4 ~ "Strong",
  growth_score <= 1 ~ "Weak",
  TRUE ~ "Moderate"
),

policy_state = case_when(
  policy_score >= 2 ~ "Restrictive",
  policy_score == 1 ~ "Neutral",
  TRUE ~ "Easy"
),

curve_state = case_when(
  curve_score >= 2 ~ "Healthy",
  curve_score == 1 ~ "Flat",
  TRUE ~ "Inverted"
),

credit_state = case_when(
  credit_score >= 2 ~ "Easy",
  credit_score == 1 ~ "Mixed",
  TRUE ~ "Tight"
),

# ---------------------------------------------------
# 7. Primary inflation-growth regime
```

```r
  # This is the main economic quadrant
  # ---------------------------------------------------
  primary_regime = case_when(
    growth_state == "Strong"   & inflation_state == "Low"      ~ "Goldilocks",
    growth_state == "Strong"   & inflation_state %in% c("Moderate", "High") ~ "Reflation",
    growth_state == "Weak"     & inflation_state == "High"     ~ "Stagflation",
    growth_state == "Weak"     & inflation_state %in% c("Low", "Moderate") ~ "Slowdown",
    growth_state == "Moderate" & inflation_state == "High"     ~ "LateCycle",
    growth_state == "Moderate" & inflation_state == "Low"      ~ "Recovery",
    TRUE ~ "Transition"
  ),

  # ---------------------------------------------------
  # 8. Risk overlay regime
  # Distinguishes benign vs stressed versions
  # of the same inflation/growth quadrant
  # ---------------------------------------------------
  risk_overlay = case_when(
    policy_state == "Restrictive" & curve_state == "Inverted" & credit_state == "Tight" ~ "Stress",
    policy_state == "Restrictive" & curve_state == "Inverted"                            ~ "Caution"
    credit_state == "Tight"                                                            ~ "Caution",
    curve_state == "Healthy" & credit_state == "Easy"                                  ~ "Supportive",
    TRUE                                                                                 ~ "Neutral"
  ),

  # ---------------------------------------------------
  # 9. Final regime label
  # ---------------------------------------------------
  regime = case_when(
    primary_regime == "Goldilocks" & risk_overlay %in% c("Supportive", "Neutral") ~ "Goldilocks",
    primary_regime == "Reflation"  & risk_overlay %in% c("Supportive", "Neutral") ~ "Reflation",
    primary_regime == "Reflation"  & risk_overlay %in% c("Caution", "Stress")     ~ "Overheat",
    primary_regime == "Slowdown"   & risk_overlay == "Supportive"                  ~ "SoftLanding",
    primary_regime == "Slowdown"   & risk_overlay %in% c("Neutral", "Caution")     ~ "Slowdown",
    primary_regime == "Slowdown"   & risk_overlay == "Stress"                      ~ "HardLanding",
    primary_regime == "Stagflation"                                                 ~ "Stagflation",
    primary_regime == "LateCycle"   & risk_overlay %in% c("Caution", "Stress")     ~ "LateCycle",
    primary_regime == "Recovery"                                                    ~ "Recovery",
    TRUE                                                                            ~ "Transition"
  ),

  # ---------------------------------------------------
  # 10. Composite macro score
  # Higher = more risk-friendly backdrop
  # Lower = more defensive backdrop
  # ---------------------------------------------------
  macro_score =
    1.25 * growth_score +
    0.75 * curve_score +
    0.75 * credit_score -
    1.00 * inflation_score -
    1.00 * policy_score,
```

```r
    macro_bucket = case_when(
      macro_score >= 3  ~ "RiskOn",
      macro_score <= -1 ~ "RiskOff",
      TRUE              ~ "Neutral"
    ),

    # ---------------------------------------------------
    # 11. Simple directional signals for continuity
    # ---------------------------------------------------
    inflation_up = if_else(inflation_state == "High", 1, 0, missing = NA_integer_),
    growth_up    = if_else(growth_state == "Strong", 1, 0, missing = NA_integer_)
  ) %>%
  arrange(date)

# -----------------------------
# Inspect result
# -----------------------------
glimpse(macro_regimes)

count(macro_regimes, regime, sort = TRUE)
count(macro_regimes, primary_regime, sort = TRUE)
count(macro_regimes, risk_overlay, sort = TRUE)
count(macro_regimes, macro_bucket, sort = TRUE)

# -----------------------------
# Save intermediate file
# -----------------------------
write_csv(macro_regimes, "macro_regimes_step2_improved.csv")



# ==========================================================
# Step 3: Pull asset prices and build monthly return table
# Improved version:
# - broader asset universe
# - safer data pull wrapper
# - standardized month-end adjusted returns
# - better mapping to macro regimes
#
# Asset sleeves:
#   SPY  = U.S. large-cap beta
#   QQQ  = growth / duration-sensitive equities
#   IWM  = small caps / domestic cyclicals
#   EFA  = developed ex-US equities
#   EEM  = emerging markets
#   IEF  = intermediate Treasuries
#   TLT  = long-duration Treasuries
#   LQD  = investment-grade credit
#   HYG  = high-yield credit
#   TIP  = inflation-linked bonds
#   GLD  = gold
#   DBC  = broad commodities
#   VNQ  = REITs / real assets
```

```r
#   BIL  = cash / T-bills
# ==========================================================

suppressPackageStartupMessages({
  library(tidyverse)
  library(lubridate)
  library(quantmod)
  library(xts)
  library(purrr)
})

Sys.setlocale("LC_TIME", "C")

# ------------------------------
# Asset universe
# ------------------------------
tickers <- c(
  "SPY", "QQQ", "IWM",
  "EFA", "EEM",
  "IEF", "TLT",
  "LQD", "HYG",
  "TIP", "GLD", "DBC", "VNQ",
  "BIL"
)

# ------------------------------
# Safer Yahoo downloader
# Returns named xts objects in env
# ------------------------------
safe_getsymbols <- function(symbols,
                            from = "2000-01-01",
                            env = .GlobalEnv,
                            max_attempts = 3) {
  for (sym in symbols) {
    success <- FALSE

    for (i in seq_len(max_attempts)) {
      out <- try(
        getSymbols(
          Symbols = sym,
          src = "yahoo",
          from = from,
          auto.assign = FALSE,
          warnings = FALSE
        ),
        silent = TRUE
      )

      if (!inherits(out, "try-error")) {
        assign(sym, out, envir = env)
        success <- TRUE
        break
      }
```

```r
    }

    if (!success) {
      warning(paste("Failed to download:", sym))
    }
  }
}

safe_getsymbols(tickers, from = "2000-01-01")

# ------------------------------
# Helper:
# Convert adjusted prices to month-end returns
# ------------------------------
monthly_returns_from_xts <- function(x, col_name) {
  px <- Ad(x)
  monthly_px <- to.monthly(px, indexAt = "lastof", OHLC = FALSE)

  tibble(
    date = as.Date(index(monthly_px)),
    !!col_name := as.numeric(monthly_px[, 1] / lag(monthly_px[, 1], 1) - 1)
  ) %>%
    arrange(date)
}

# ------------------------------
# Build monthly return tables
# Only create table if object exists
# ------------------------------
asset_return_list <- list(
  if (exists("SPY")) monthly_returns_from_xts(SPY, "spy_ret"),
  if (exists("QQQ")) monthly_returns_from_xts(QQQ, "qqq_ret"),
  if (exists("IWM")) monthly_returns_from_xts(IWM, "iwm_ret"),
  if (exists("EFA")) monthly_returns_from_xts(EFA, "efa_ret"),
  if (exists("EEM")) monthly_returns_from_xts(EEM, "eem_ret"),
  if (exists("IEF")) monthly_returns_from_xts(IEF, "ief_ret"),
  if (exists("TLT")) monthly_returns_from_xts(TLT, "tlt_ret"),
  if (exists("LQD")) monthly_returns_from_xts(LQD, "lqd_ret"),
  if (exists("HYG")) monthly_returns_from_xts(HYG, "hyg_ret"),
  if (exists("TIP")) monthly_returns_from_xts(TIP, "tip_ret"),
  if (exists("GLD")) monthly_returns_from_xts(GLD, "gld_ret"),
  if (exists("DBC")) monthly_returns_from_xts(DBC, "dbc_ret"),
  if (exists("VNQ")) monthly_returns_from_xts(VNQ, "vnq_ret"),
  if (exists("BIL")) monthly_returns_from_xts(BIL, "bil_ret")
) %>%
  compact()

# ------------------------------
# Merge all monthly return tables
# ------------------------------
asset_returns <- reduce(asset_return_list, full_join, by = "date") %>%
  arrange(date) %>%
  mutate(date = ceiling_date(date, "month") - days(1)) %>%
```

```r
  filter(date >= as.Date("2004-01-31"))

# ----------------------------
# Optional: sleeve composites
# These help simplify later weight design
# ----------------------------
asset_returns <- asset_returns %>%
  mutate(
    us_equity_ret = rowMeans(select(., spy_ret, qqq_ret, iwm_ret), na.rm = TRUE),
    intl_equity_ret = rowMeans(select(., efa_ret, eem_ret), na.rm = TRUE),
    equity_ret = rowMeans(select(., spy_ret, qqq_ret, iwm_ret, efa_ret, eem_ret), na.rm = TRUE),

    treasury_ret = rowMeans(select(., ief_ret, tlt_ret), na.rm = TRUE),
    credit_ret   = rowMeans(select(., lqd_ret, hyg_ret), na.rm = TRUE),
    real_asset_ret = rowMeans(select(., tip_ret, gld_ret, dbc_ret, vnq_ret), na.rm = TRUE)
  )

# ----------------------------
# Optional: availability flags
# Useful later if you want to exclude
# periods before ETF inception from a sleeve
# ----------------------------
asset_returns <- asset_returns %>%
  mutate(
    n_equity_assets = rowSums(!is.na(select(., spy_ret, qqq_ret, iwm_ret, efa_ret, eem_ret))),
    n_treasury_assets = rowSums(!is.na(select(., ief_ret, tlt_ret))),
    n_credit_assets = rowSums(!is.na(select(., lqd_ret, hyg_ret))),
    n_real_assets = rowSums(!is.na(select(., tip_ret, gld_ret, dbc_ret, vnq_ret)))
  )

# ----------------------------
# Inspect
# ----------------------------
glimpse(asset_returns)
print(head(asset_returns, 15))
print(tail(asset_returns, 15))

# ----------------------------
# Save intermediate file
# ----------------------------
write_csv(asset_returns, "asset_returns_step3_improved.csv")



# ==========================================================
# Step 4: Join lagged macro regimes to asset returns,
# build dynamic regime weights, and compute backtest returns
# Improved version:
# - one-month lagged signals
# - expanded asset universe
# - regime-specific weights
# - automatic renormalization when some ETFs are unavailable
# ==========================================================
```

```r
# ------------------------------
# 4A. Align macro and asset data at month-end
# ------------------------------
macro_regimes_aligned <- macro_regimes %>%
  mutate(date = ceiling_date(date, "month") - days(1)) %>%
  arrange(date)

asset_returns_aligned <- asset_returns %>%
  mutate(date = ceiling_date(date, "month") - days(1)) %>%
  arrange(date)

# ------------------------------
# 4B. Create model table with lagged macro signals
# The portfolio formed for month t uses signals from month t-1
# ------------------------------
model_data <- macro_regimes_aligned %>%
  transmute(
    date,
    regime_signal       = lag(regime, 1),
    primary_signal      = lag(primary_regime, 1),
    overlay_signal      = lag(risk_overlay, 1),
    macro_bucket_signal = lag(macro_bucket, 1),
    macro_score_signal  = lag(macro_score, 1),
    inflation_signal    = lag(inflation_up, 1),
    growth_signal       = lag(growth_up, 1)
  ) %>%
  inner_join(asset_returns_aligned, by = "date") %>%
  arrange(date)

glimpse(model_data)
print(head(model_data, 20))
print(tail(model_data, 20))

write_csv(model_data, "model_data_step4_improved.csv")

# ------------------------------
# 4C. Regime-based target weights
# Weights are set at the ETF level
# They should sum to 1 within each regime
# ------------------------------
regime_weights <- tribble(
  ~regime_signal, ~w_spy, ~w_qqq, ~w_iwm, ~w_efa, ~w_eem, ~w_ief, ~w_tlt, ~w_lqd, ~w_hyg, ~w_tip, ~w_gl

  # Strong growth, tame inflation, supportive backdrop
  "Goldilocks",    0.22,   0.20,   0.08,   0.10,   0.05,   0.05,   0.00,   0.07,   0.10,   0.03,   0.03

  # Growth accelerating, inflation firming
  "Reflation",     0.16,   0.08,   0.14,   0.08,   0.10,   0.03,   0.00,   0.04,   0.10,   0.06,   0.08

  # Growth still decent but inflation/policy pressure is becoming a problem
  "Overheat",      0.10,   0.04,   0.08,   0.06,   0.06,   0.04,   0.00,   0.04,   0.06,   0.12,   0.14

  # Benign slowdown, not yet full stress
```

```r
  "SoftLanding",   0.18,   0.10,   0.05,   0.08,   0.04,   0.12,   0.05,   0.08,   0.07,   0.06,   0.05

  # Clear slowdown, more defense
  "Slowdown",      0.10,   0.05,   0.03,   0.05,   0.02,   0.18,   0.14,   0.08,   0.03,   0.07,   0.07

  # Recession / stress / hard landing
  "HardLanding",   0.04,   0.02,   0.01,   0.02,   0.00,   0.22,   0.24,   0.05,   0.00,   0.05,   0.10

  # Weak growth + high inflation
  "Stagflation",   0.04,   0.01,   0.02,   0.02,   0.02,   0.05,   0.00,   0.02,   0.02,   0.16,   0.24

  # Mid-cycle recovery from weakness
  "Recovery",      0.20,   0.12,   0.10,   0.09,   0.06,   0.08,   0.02,   0.07,   0.09,   0.04,   0.04

  # Ambiguous state
  "Transition",    0.14,   0.08,   0.05,   0.07,   0.03,   0.12,   0.06,   0.08,   0.05,   0.06,   0.07

  # Late-cycle but not yet broken
  "LateCycle",     0.11,   0.05,   0.06,   0.06,   0.04,   0.08,   0.03,   0.06,   0.05,   0.10,   0.12
)

# Quick integrity check
weight_check <- regime_weights %>%
  mutate(weight_sum = w_spy + w_qqq + w_iwm + w_efa + w_eem + w_ief + w_tlt +
           w_lqd + w_hyg + w_tip + w_gld + w_dbc + w_vnq + w_bil)

print(weight_check %>% select(regime_signal, weight_sum))

# ----------------------------
# 4D. Join weights to model data
# ----------------------------
backtest_data_raw <- model_data %>%
  left_join(regime_weights, by = "regime_signal") %>%
  arrange(date)

# ----------------------------
# 4E. Normalize weights over assets that actually exist that month
# This avoids broken early-history months where some ETFs had not launched yet
# ----------------------------
backtest_data <- backtest_data_raw %>%
  mutate(
    avail_spy = if_else(!is.na(spy_ret), 1, 0),
    avail_qqq = if_else(!is.na(qqq_ret), 1, 0),
    avail_iwm = if_else(!is.na(iwm_ret), 1, 0),
    avail_efa = if_else(!is.na(efa_ret), 1, 0),
    avail_eem = if_else(!is.na(eem_ret), 1, 0),
    avail_ief = if_else(!is.na(ief_ret), 1, 0),
    avail_tlt = if_else(!is.na(tlt_ret), 1, 0),
    avail_lqd = if_else(!is.na(lqd_ret), 1, 0),
    avail_hyg = if_else(!is.na(hyg_ret), 1, 0),
    avail_tip = if_else(!is.na(tip_ret), 1, 0),
    avail_gld = if_else(!is.na(gld_ret), 1, 0),
    avail_dbc = if_else(!is.na(dbc_ret), 1, 0),
```

```r
  avail_vnq = if_else(!is.na(vnq_ret), 1, 0),
  avail_bil = if_else(!is.na(bil_ret), 1, 0),

  raw_weight_sum =
    w_spy * avail_spy +
    w_qqq * avail_qqq +
    w_iwm * avail_iwm +
    w_efa * avail_efa +
    w_eem * avail_eem +
    w_ief * avail_ief +
    w_tlt * avail_tlt +
    w_lqd * avail_lqd +
    w_hyg * avail_hyg +
    w_tip * avail_tip +
    w_gld * avail_gld +
    w_dbc * avail_dbc +
    w_vnq * avail_vnq +
    w_bil * avail_bil,

  w_spy_n = if_else(raw_weight_sum > 0, w_spy * avail_spy / raw_weight_sum, NA_real_),
  w_qqq_n = if_else(raw_weight_sum > 0, w_qqq * avail_qqq / raw_weight_sum, NA_real_),
  w_iwm_n = if_else(raw_weight_sum > 0, w_iwm * avail_iwm / raw_weight_sum, NA_real_),
  w_efa_n = if_else(raw_weight_sum > 0, w_efa * avail_efa / raw_weight_sum, NA_real_),
  w_eem_n = if_else(raw_weight_sum > 0, w_eem * avail_eem / raw_weight_sum, NA_real_),
  w_ief_n = if_else(raw_weight_sum > 0, w_ief * avail_ief / raw_weight_sum, NA_real_),
  w_tlt_n = if_else(raw_weight_sum > 0, w_tlt * avail_tlt / raw_weight_sum, NA_real_),
  w_lqd_n = if_else(raw_weight_sum > 0, w_lqd * avail_lqd / raw_weight_sum, NA_real_),
  w_hyg_n = if_else(raw_weight_sum > 0, w_hyg * avail_hyg / raw_weight_sum, NA_real_),
  w_tip_n = if_else(raw_weight_sum > 0, w_tip * avail_tip / raw_weight_sum, NA_real_),
  w_gld_n = if_else(raw_weight_sum > 0, w_gld * avail_gld / raw_weight_sum, NA_real_),
  w_dbc_n = if_else(raw_weight_sum > 0, w_dbc * avail_dbc / raw_weight_sum, NA_real_),
  w_vnq_n = if_else(raw_weight_sum > 0, w_vnq * avail_vnq / raw_weight_sum, NA_real_),
  w_bil_n = if_else(raw_weight_sum > 0, w_bil * avail_bil / raw_weight_sum, NA_real_),

  strategy_ret =
    coalesce(w_spy_n, 0) * coalesce(spy_ret, 0) +
    coalesce(w_qqq_n, 0) * coalesce(qqq_ret, 0) +
    coalesce(w_iwm_n, 0) * coalesce(iwm_ret, 0) +
    coalesce(w_efa_n, 0) * coalesce(efa_ret, 0) +
    coalesce(w_eem_n, 0) * coalesce(eem_ret, 0) +
    coalesce(w_ief_n, 0) * coalesce(ief_ret, 0) +
    coalesce(w_tlt_n, 0) * coalesce(tlt_ret, 0) +
    coalesce(w_lqd_n, 0) * coalesce(lqd_ret, 0) +
    coalesce(w_hyg_n, 0) * coalesce(hyg_ret, 0) +
    coalesce(w_tip_n, 0) * coalesce(tip_ret, 0) +
    coalesce(w_gld_n, 0) * coalesce(gld_ret, 0) +
    coalesce(w_dbc_n, 0) * coalesce(dbc_ret, 0) +
    coalesce(w_vnq_n, 0) * coalesce(vnq_ret, 0) +
    coalesce(w_bil_n, 0) * coalesce(bil_ret, 0),

  # Benchmarks
  benchmark_spy_ret   = spy_ret,
  benchmark_6040_ret  = 0.60 * coalesce(spy_ret, 0) + 0.40 * coalesce(ief_ret, 0),
```

```r
    benchmark_allweather_ret =
      0.30 * coalesce(spy_ret, 0) +
      0.40 * coalesce(tlt_ret, 0) +
      0.15 * coalesce(tip_ret, 0) +
      0.075 * coalesce(gld_ret, 0) +
      0.075 * coalesce(dbc_ret, 0)
  ) %>%
  filter(
    !is.na(regime_signal),
    !is.na(strategy_ret),
    !is.na(benchmark_spy_ret)
  ) %>%
  arrange(date)

glimpse(backtest_data)
print(head(backtest_data, 20))
print(tail(backtest_data, 20))

write_csv(backtest_data, "backtest_data_step4_improved.csv")



# ==========================================================
# Step 7: Build cumulative performance series
# Includes gross and transaction-cost-adjusted paths
# ==========================================================

# ------------------------------
# 7A. Turnover and transaction cost model
# Assumption:
# - turnover = 0.5 * sum(abs(w_t - w_{t-1}))
# - tc_cost = turnover * one_way_cost
# You can change one_way_cost to test sensitivity
# ------------------------------
one_way_cost <- 0.0010   # 10 bps one-way trading cost

backtest_data <- backtest_data %>%
  arrange(date) %>%
  mutate(
    turnover =
      0.5 * (
        abs(w_spy_n - lag(w_spy_n)) +
        abs(w_qqq_n - lag(w_qqq_n)) +
        abs(w_iwm_n - lag(w_iwm_n)) +
        abs(w_efa_n - lag(w_efa_n)) +
        abs(w_eem_n - lag(w_eem_n)) +
        abs(w_ief_n - lag(w_ief_n)) +
        abs(w_tlt_n - lag(w_tlt_n)) +
        abs(w_lqd_n - lag(w_lqd_n)) +
        abs(w_hyg_n - lag(w_hyg_n)) +
        abs(w_tip_n - lag(w_tip_n)) +
        abs(w_gld_n - lag(w_gld_n)) +
        abs(w_dbc_n - lag(w_dbc_n)) +
```

```r
        abs(w_vnq_n - lag(w_vnq_n)) +
        abs(w_bil_n - lag(w_bil_n))
      ),

    turnover = replace_na(turnover, 0),
    tc_cost = turnover * one_way_cost,
    strategy_ret_net = strategy_ret - tc_cost
  )

# ----------------------------
# 7B. Cumulative index paths
# ----------------------------
performance_path <- backtest_data %>%
  arrange(date) %>%
  mutate(
    strategy_index_gross = 100 * cumprod(1 + strategy_ret),
    strategy_index_net   = 100 * cumprod(1 + strategy_ret_net),
    spy_index            = 100 * cumprod(1 + benchmark_spy_ret),
    bal6040_index        = 100 * cumprod(1 + benchmark_6040_ret),
    allweather_index     = 100 * cumprod(1 + benchmark_allweather_ret)
  )

glimpse(performance_path)
print(head(performance_path, 20))
print(tail(performance_path, 20))

write_csv(performance_path, "performance_path_step7_improved.csv")



# ==========================================================
# Step 8: Plot cumulative performance
# ==========================================================

plot_data <- performance_path %>%
  select(
    date,
    strategy_index_gross,
    strategy_index_net,
    spy_index,
    bal6040_index,
    allweather_index
  ) %>%
  pivot_longer(
    cols = -date,
    names_to = "series",
    values_to = "index_value"
  )

ggplot(plot_data, aes(x = date, y = index_value, color = series)) +
  geom_line(linewidth = 1) +
  labs(
    title = "Macro Regime Allocation Model vs Benchmarks",
```

```r
    x = NULL,
    y = "Growth of 100",
    color = NULL
  ) +
  theme_minimal(base_size = 12)



# ==========================================================
# Step 9: Performance metrics
# Expanded performance statistics
# ==========================================================

perf_stats <- function(ret_vec) {
  ret_vec <- ret_vec[is.finite(ret_vec)]

  n_months <- length(ret_vec)

  ann_return <- (prod(1 + ret_vec)^(12 / n_months)) - 1
  ann_vol    <- sd(ret_vec) * sqrt(12)
  sharpe     <- ifelse(ann_vol == 0, NA_real_, ann_return / ann_vol)

  wealth      <- cumprod(1 + ret_vec)
  drawdown    <- wealth / cummax(wealth) - 1
  max_dd      <- min(drawdown, na.rm = TRUE)

  downside_sd <- sd(if_else(ret_vec < 0, ret_vec, 0), na.rm = TRUE) * sqrt(12)
  sortino     <- ifelse(downside_sd == 0, NA_real_, ann_return / downside_sd)

  calmar      <- ifelse(max_dd == 0, NA_real_, ann_return / abs(max_dd))
  hit_rate    <- mean(ret_vec > 0, na.rm = TRUE)
  best_month  <- max(ret_vec, na.rm = TRUE)
  worst_month <- min(ret_vec, na.rm = TRUE)

  tibble(
    Months = n_months,
    CAGR = ann_return,
    AnnVol = ann_vol,
    Sharpe = sharpe,
    Sortino = sortino,
    Calmar = calmar,
    MaxDrawdown = max_dd,
    HitRate = hit_rate,
    BestMonth = best_month,
    WorstMonth = worst_month
  )
}

stats_strategy_gross <- perf_stats(performance_path$strategy_ret) %>%
  mutate(series = "Strategy_Gross")

stats_strategy_net <- perf_stats(performance_path$strategy_ret_net) %>%
  mutate(series = "Strategy_Net")
```

```r
stats_spy <- perf_stats(performance_path$benchmark_spy_ret) %>%
  mutate(series = "SPY")

stats_6040 <- perf_stats(performance_path$benchmark_6040_ret) %>%
  mutate(series = "60_40")

stats_allweather <- perf_stats(performance_path$benchmark_allweather_ret) %>%
  mutate(series = "AllWeather")

stats_table <- bind_rows(
  stats_strategy_gross,
  stats_strategy_net,
  stats_spy,
  stats_6040,
  stats_allweather
) %>%
  select(series, everything())

print(stats_table)
write_csv(stats_table, "performance_stats_step9_improved.csv")



# ==========================================================
# Step 10: Add drawdown paths
# ==========================================================

performance_path <- performance_path %>%
  mutate(
    strategy_dd_gross = strategy_index_gross / cummax(strategy_index_gross) - 1,
    strategy_dd_net   = strategy_index_net / cummax(strategy_index_net) - 1,
    spy_dd            = spy_index / cummax(spy_index) - 1,
    bal6040_dd        = bal6040_index / cummax(bal6040_index) - 1,
    allweather_dd     = allweather_index / cummax(allweather_index) - 1
  )

write_csv(performance_path, "performance_path_step10_improved.csv")



# ==========================================================
# Step 11: Plot drawdowns
# ==========================================================

drawdown_plot_data <- performance_path %>%
  select(
    date,
    strategy_dd_gross,
    strategy_dd_net,
    spy_dd,
    bal6040_dd,
    allweather_dd
  ) %>%
```

```r
  pivot_longer(
    cols = -date,
    names_to = "series",
    values_to = "drawdown"
  )

ggplot(drawdown_plot_data, aes(x = date, y = drawdown, color = series)) +
  geom_line(linewidth = 1) +
  labs(
    title = "Drawdowns: Strategy vs Benchmarks",
    x = NULL,
    y = "Drawdown",
    color = NULL
  ) +
  theme_minimal(base_size = 12)



# ==========================================================
# Step 12: Regime diagnostics
# How often does each regime occur?
# What were average returns inside each regime?
# ==========================================================

regime_summary <- backtest_data %>%
  group_by(regime_signal) %>%
  summarise(
    months = n(),

    avg_strategy_ret_gross = mean(strategy_ret, na.rm = TRUE),
    avg_strategy_ret_net   = mean(strategy_ret_net, na.rm = TRUE),

    avg_spy_ret = mean(spy_ret, na.rm = TRUE),
    avg_qqq_ret = mean(qqq_ret, na.rm = TRUE),
    avg_iwm_ret = mean(iwm_ret, na.rm = TRUE),
    avg_efa_ret = mean(efa_ret, na.rm = TRUE),
    avg_eem_ret = mean(eem_ret, na.rm = TRUE),
    avg_ief_ret = mean(ief_ret, na.rm = TRUE),
    avg_tlt_ret = mean(tlt_ret, na.rm = TRUE),
    avg_lqd_ret = mean(lqd_ret, na.rm = TRUE),
    avg_hyg_ret = mean(hyg_ret, na.rm = TRUE),
    avg_tip_ret = mean(tip_ret, na.rm = TRUE),
    avg_gld_ret = mean(gld_ret, na.rm = TRUE),
    avg_dbc_ret = mean(dbc_ret, na.rm = TRUE),
    avg_vnq_ret = mean(vnq_ret, na.rm = TRUE),
    avg_bil_ret = mean(bil_ret, na.rm = TRUE),

    avg_turnover = mean(turnover, na.rm = TRUE),
    strategy_minus_spy = mean(strategy_ret - benchmark_spy_ret, na.rm = TRUE),
    strategy_minus_6040 = mean(strategy_ret - benchmark_6040_ret, na.rm = TRUE),

    .groups = "drop"
  ) %>%
```

```r
  arrange(desc(months))

print(regime_summary)
write_csv(regime_summary, "regime_summary_step12_improved.csv")



# ==========================================================
# Step 13: Rolling 12-month returns and rolling relative returns
# ==========================================================

performance_path <- performance_path %>%
  mutate(
    strategy_12m_gross = strategy_index_gross / lag(strategy_index_gross, 12) - 1,
    strategy_12m_net   = strategy_index_net / lag(strategy_index_net, 12) - 1,
    spy_12m            = spy_index / lag(spy_index, 12) - 1,
    bal6040_12m        = bal6040_index / lag(bal6040_index, 12) - 1,
    allweather_12m     = allweather_index / lag(allweather_index, 12) - 1,

    rel_12m_vs_spy     = strategy_12m_net - spy_12m,
    rel_12m_vs_6040    = strategy_12m_net - bal6040_12m,
    rel_12m_vs_aw      = strategy_12m_net - allweather_12m
  )

write_csv(performance_path, "performance_path_step13_improved.csv")



# ==========================================================
# Step 14A: Plot rolling 12-month returns
# ==========================================================

rolling_plot_data <- performance_path %>%
  select(
    date,
    strategy_12m_gross,
    strategy_12m_net,
    spy_12m,
    bal6040_12m,
    allweather_12m
  ) %>%
  pivot_longer(
    cols = -date,
    names_to = "series",
    values_to = "rolling_return"
  ) %>%
  filter(!is.na(rolling_return))

ggplot(rolling_plot_data, aes(x = date, y = rolling_return, color = series)) +
  geom_line(linewidth = 1) +
  labs(
    title = "Rolling 12-Month Returns",
    x = NULL,
```

```r
    y = "Return",
    color = NULL
  ) +
  theme_minimal(base_size = 12)



# ==========================================================
# Step 14B: Plot rolling relative performance
# ==========================================================

rolling_relative_plot_data <- performance_path %>%
  select(date, rel_12m_vs_spy, rel_12m_vs_6040, rel_12m_vs_aw) %>%
  pivot_longer(
    cols = -date,
    names_to = "series",
    values_to = "relative_return"
  ) %>%
  filter(!is.na(relative_return))

ggplot(rolling_relative_plot_data, aes(x = date, y = relative_return, color = series)) +
  geom_line(linewidth = 1) +
  geom_hline(yintercept = 0, linewidth = 0.5) +
  labs(
    title = "Rolling 12-Month Relative Performance",
    x = NULL,
    y = "Strategy minus Benchmark",
    color = NULL
  ) +
  theme_minimal(base_size = 12)



# ==========================================================
# Step 15: Subperiod performance
# ==========================================================

subperiod_stats <- performance_path %>%
  mutate(
    period = case_when(
      date < as.Date("2008-01-01") ~ "2004-2007",
      date < as.Date("2013-01-01") ~ "2008-2012",
      date < as.Date("2020-01-01") ~ "2013-2019",
      TRUE                         ~ "2020+"
    )
  ) %>%
  group_by(period) %>%
  summarise(
    strategy_cagr_gross = (last(strategy_index_gross) / first(strategy_index_gross))^(12 / n()) - 1,
    strategy_cagr_net   = (last(strategy_index_net) / first(strategy_index_net))^(12 / n()) - 1,
    spy_cagr            = (last(spy_index) / first(spy_index))^(12 / n()) - 1,
    bal6040_cagr        = (last(bal6040_index) / first(bal6040_index))^(12 / n()) - 1,
    allweather_cagr     = (last(allweather_index) / first(allweather_index))^(12 / n()) - 1,
```

```r
    avg_turnover        = mean(turnover, na.rm = TRUE),
    .groups = "drop"
  )

print(subperiod_stats)
write_csv(subperiod_stats, "subperiod_stats_step15.csv")



# ==========================================================
# Step 16: Regime transition and turnover diagnostics
# ==========================================================

regime_transition_stats <- backtest_data %>%
  mutate(
    prev_regime = lag(regime_signal),
    turnover_flag = if_else(regime_signal != prev_regime, 1, 0, missing = 0)
  ) %>%
  summarise(
    avg_monthly_turnover = mean(turnover, na.rm = TRUE),
    median_monthly_turnover = median(turnover, na.rm = TRUE),
    max_monthly_turnover = max(turnover, na.rm = TRUE),
    regime_switch_rate = mean(turnover_flag, na.rm = TRUE),
    avg_tc_cost = mean(tc_cost, na.rm = TRUE)
  )

print(regime_transition_stats)
write_csv(regime_transition_stats, "regime_transition_stats_step16.csv")



# ==========================================================
# Step 17: Average exposures by regime
# ==========================================================

regime_exposure_summary <- backtest_data %>%
  group_by(regime_signal) %>%
  summarise(
    w_spy = mean(w_spy_n, na.rm = TRUE),
    w_qqq = mean(w_qqq_n, na.rm = TRUE),
    w_iwm = mean(w_iwm_n, na.rm = TRUE),
    w_efa = mean(w_efa_n, na.rm = TRUE),
    w_eem = mean(w_eem_n, na.rm = TRUE),
    w_ief = mean(w_ief_n, na.rm = TRUE),
    w_tlt = mean(w_tlt_n, na.rm = TRUE),
    w_lqd = mean(w_lqd_n, na.rm = TRUE),
    w_hyg = mean(w_hyg_n, na.rm = TRUE),
    w_tip = mean(w_tip_n, na.rm = TRUE),
    w_gld = mean(w_gld_n, na.rm = TRUE),
    w_dbc = mean(w_dbc_n, na.rm = TRUE),
    w_vnq = mean(w_vnq_n, na.rm = TRUE),
    w_bil = mean(w_bil_n, na.rm = TRUE),
    .groups = "drop"
```

```r
  )

print(regime_exposure_summary)
write_csv(regime_exposure_summary, "regime_exposure_summary_step17.csv")



# ==========================================================
# Step 18: Plot allocation history
# ==========================================================

allocation_plot_data <- backtest_data %>%
  select(
    date,
    w_spy_n, w_qqq_n, w_iwm_n, w_efa_n, w_eem_n,
    w_ief_n, w_tlt_n, w_lqd_n, w_hyg_n,
    w_tip_n, w_gld_n, w_dbc_n, w_vnq_n, w_bil_n
  ) %>%
  pivot_longer(
    cols = -date,
    names_to = "asset",
    values_to = "weight"
  )

ggplot(allocation_plot_data, aes(x = date, y = weight, color = asset)) +
  geom_line(linewidth = 0.9) +
  labs(
    title = "Dynamic Portfolio Weights Through Time",
    x = NULL,
    y = "Portfolio Weight",
    color = NULL
  ) +
  theme_minimal(base_size = 12)



# ==========================================================
# Step 19: Sleeve contribution analysis
# ==========================================================

contribution_data <- backtest_data %>%
  mutate(
    contrib_us_equity =
      coalesce(w_spy_n, 0) * coalesce(spy_ret, 0) +
      coalesce(w_qqq_n, 0) * coalesce(qqq_ret, 0) +
      coalesce(w_iwm_n, 0) * coalesce(iwm_ret, 0),

    contrib_intl_equity =
      coalesce(w_efa_n, 0) * coalesce(efa_ret, 0) +
      coalesce(w_eem_n, 0) * coalesce(eem_ret, 0),

    contrib_treasury =
      coalesce(w_ief_n, 0) * coalesce(ief_ret, 0) +
```

```r
      coalesce(w_tlt_n, 0) * coalesce(tlt_ret, 0),

    contrib_credit =
      coalesce(w_lqd_n, 0) * coalesce(lqd_ret, 0) +
      coalesce(w_hyg_n, 0) * coalesce(hyg_ret, 0),

    contrib_real_assets =
      coalesce(w_tip_n, 0) * coalesce(tip_ret, 0) +
      coalesce(w_gld_n, 0) * coalesce(gld_ret, 0) +
      coalesce(w_dbc_n, 0) * coalesce(dbc_ret, 0) +
      coalesce(w_vnq_n, 0) * coalesce(vnq_ret, 0),

    contrib_cash =
      coalesce(w_bil_n, 0) * coalesce(bil_ret, 0)
  )

contribution_summary <- contribution_data %>%
  summarise(
    us_equity = mean(contrib_us_equity, na.rm = TRUE),
    intl_equity = mean(contrib_intl_equity, na.rm = TRUE),
    treasury = mean(contrib_treasury, na.rm = TRUE),
    credit = mean(contrib_credit, na.rm = TRUE),
    real_assets = mean(contrib_real_assets, na.rm = TRUE),
    cash = mean(contrib_cash, na.rm = TRUE)
  ) %>%
  pivot_longer(
    cols = everything(),
    names_to = "sleeve",
    values_to = "avg_monthly_contribution"
  )

print(contribution_summary)
write_csv(contribution_summary, "contribution_summary_step19.csv")



# ==========================================================
# Step 20: Regime hit-rate diagnostics
# Did the chosen regime beat SPY / 60-40 in that month?
# ==========================================================

regime_hit_rates <- backtest_data %>%
  mutate(
    beat_spy = if_else(strategy_ret_net > benchmark_spy_ret, 1, 0, missing = NA_real_),
    beat_6040 = if_else(strategy_ret_net > benchmark_6040_ret, 1, 0, missing = NA_real_),
    beat_aw = if_else(strategy_ret_net > benchmark_allweather_ret, 1, 0, missing = NA_real_)
  ) %>%
  group_by(regime_signal) %>%
  summarise(
    months = n(),
    hit_rate_vs_spy = mean(beat_spy, na.rm = TRUE),
    hit_rate_vs_6040 = mean(beat_6040, na.rm = TRUE),
    hit_rate_vs_aw = mean(beat_aw, na.rm = TRUE),
```

```r
    avg_alpha_vs_spy = mean(strategy_ret_net - benchmark_spy_ret, na.rm = TRUE),
    avg_alpha_vs_6040 = mean(strategy_ret_net - benchmark_6040_ret, na.rm = TRUE),
    avg_alpha_vs_aw = mean(strategy_ret_net - benchmark_allweather_ret, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(months))

print(regime_hit_rates)
write_csv(regime_hit_rates, "regime_hit_rates_step20.csv")
```

```r
# ==========================================================
# V2 Step 1: Build continuous macro + momentum signal set
# Purpose:
# - reuse V1 macro features
# - convert them into comparable continuous signals
# - add forward-looking market momentum inputs
# ==========================================================

suppressPackageStartupMessages({
  library(tidyverse)
  library(lubridate)
  library(zoo)
  library(quantmod)
  library(xts)
})

Sys.setlocale("LC_TIME", "C")

# ----------------------------------------------------------
# Helper: rolling z-score using trailing window
# Uses only information available up to time t
# ----------------------------------------------------------
roll_zscore <- function(x, window = 36) {
  rollapplyr(
    data = x,
    width = window,
    FUN = function(y) {
      s <- sd(y, na.rm = TRUE)
      m <- mean(y, na.rm = TRUE)
      last_y <- tail(y, 1)

      if (is.na(s) || s == 0) {
        return(NA_real_)
      } else {
        return((last_y - m) / s)
      }
    },
    fill = NA,
    partial = FALSE
  )
}

# ----------------------------------------------------------
# Momentum helper
```

```r
# 12-1 momentum:
# cumulative return from t-12 to t-1
# Excludes most recent month to reduce short-term reversal effects
# ---------------------------------------------------------
calc_12_1_mom <- function(ret) {
  lag(
    rollapplyr(
      1 + ret,
      width = 11,
      FUN = function(x) prod(x, na.rm = FALSE),
      fill = NA,
      partial = FALSE
    ) - 1,
    1
  )
}

# ---------------------------------------------------------
# Reuse V1 macro table
# Assumes macro_monthly already exists from V1
# ---------------------------------------------------------
v2_macro_base <- macro_monthly %>%
  arrange(date) %>%
  select(
    date,

    # Inflation
    cpi_yoy,
    core_cpi_yoy,
    cpi_3m_ann,
    core_cpi_3m_ann,
    cpi_yoy_6m_delta_smooth,
    core_cpi_yoy_6m_delta_smooth,

    # Growth / labor
    unrate_6m_delta_smooth,
    payems_3m_ann_smooth,
    indpro_3m_ann_smooth,
    houst_yoy_smooth,
    umcsent_6m_delta,

    # Policy / curve / credit
    real_fedfunds,
    real_10y,
    term_spread_10y3m,
    term_spread_10y2y,
    front_end_spread_2y3m,
    baa_aaa_spread,
    baa_10y_spread_smooth
  )

# ---------------------------------------------------------
# Build continuous macro z-scores
```

```r
# Sign convention:
# Positive = more risk-friendly unless explicitly inflationary
# Inflation remains its own block for later mapping
# ----------------------------------------------------------
v2_macro_signals <- v2_macro_base %>%
  mutate(
    # -------------------------
    # Inflation block
    # Higher = hotter inflation
    # -------------------------
    z_cpi_yoy               = roll_zscore(cpi_yoy, 36),
    z_core_cpi_yoy          = roll_zscore(core_cpi_yoy, 36),
    z_cpi_3m_ann            = roll_zscore(cpi_3m_ann, 36),
    z_core_cpi_3m_ann       = roll_zscore(core_cpi_3m_ann, 36),
    z_cpi_yoy_6m_delta      = roll_zscore(cpi_yoy_6m_delta_smooth, 36),
    z_core_cpi_yoy_6m_delta = roll_zscore(core_cpi_yoy_6m_delta_smooth, 36),

    # -------------------------
    # Growth block
    # For unemployment delta:
    # falling unemployment is good, so invert sign
    # -------------------------
    z_unrate_trend     = -roll_zscore(unrate_6m_delta_smooth, 36),
    z_payems_growth    = roll_zscore(payems_3m_ann_smooth, 36),
    z_indpro_growth    = roll_zscore(indpro_3m_ann_smooth, 36),
    z_housing_growth   = roll_zscore(houst_yoy_smooth, 36),
    z_sentiment_change = roll_zscore(umcsent_6m_delta, 36),

    # -------------------------
    # Policy block
    # Higher real rates = more restrictive, so invert for risk
    # -------------------------
    z_real_fedfunds_risk = -roll_zscore(real_fedfunds, 36),
    z_real_10y_risk      = -roll_zscore(real_10y, 36),

    # -------------------------
    # Curve block
    # Steeper / more normal curve = more risk-friendly
    # -------------------------
    z_curve_10y3m = roll_zscore(term_spread_10y3m, 36),
    z_curve_10y2y = roll_zscore(term_spread_10y2y, 36),
    z_curve_2y3m  = roll_zscore(front_end_spread_2y3m, 36),

    # -------------------------
    # Credit block
    # Wider spreads = worse, so invert sign
    # -------------------------
    z_baa_aaa_risk = -roll_zscore(baa_aaa_spread, 36),
    z_baa_10y_risk = -roll_zscore(baa_10y_spread_smooth, 36)
  ) %>%
  arrange(date)

# ----------------------------------------------------------
```

```r
# Reuse V1 asset return table
# Assumes asset_returns already exists from V1 Step 3
# ----------------------------------------------------------
v2_momentum_signals <- asset_returns %>%
  arrange(date) %>%
  mutate(
    # Single-asset momentum
    mom_spy_12_1 = calc_12_1_mom(spy_ret),
    mom_qqq_12_1 = calc_12_1_mom(qqq_ret),
    mom_iwm_12_1 = calc_12_1_mom(iwm_ret),
    mom_efa_12_1 = calc_12_1_mom(efa_ret),
    mom_eem_12_1 = calc_12_1_mom(eem_ret),
    mom_ief_12_1 = calc_12_1_mom(ief_ret),
    mom_tlt_12_1 = calc_12_1_mom(tlt_ret),
    mom_lqd_12_1 = calc_12_1_mom(lqd_ret),
    mom_hyg_12_1 = calc_12_1_mom(hyg_ret),
    mom_tip_12_1 = calc_12_1_mom(tip_ret),
    mom_gld_12_1 = calc_12_1_mom(gld_ret),
    mom_dbc_12_1 = calc_12_1_mom(dbc_ret),
    mom_vnq_12_1 = calc_12_1_mom(vnq_ret),
    mom_bil_12_1 = calc_12_1_mom(bil_ret)
  ) %>%
  mutate(
    # Sleeve-level momentum
    mom_us_equity_12_1 = rowMeans(
      across(c(mom_spy_12_1, mom_qqq_12_1, mom_iwm_12_1)),
      na.rm = TRUE
    ),

    mom_intl_equity_12_1 = rowMeans(
      across(c(mom_efa_12_1, mom_eem_12_1)),
      na.rm = TRUE
    ),

    mom_treasury_12_1 = rowMeans(
      across(c(mom_ief_12_1, mom_tlt_12_1)),
      na.rm = TRUE
    ),

    mom_credit_12_1 = rowMeans(
      across(c(mom_lqd_12_1, mom_hyg_12_1)),
      na.rm = TRUE
    ),

    mom_real_assets_12_1 = rowMeans(
      across(c(mom_tip_12_1, mom_gld_12_1, mom_dbc_12_1, mom_vnq_12_1)),
      na.rm = TRUE
    )
  ) %>%
  mutate(
    across(starts_with("mom_"), ~na_if(., NaN))
  ) %>%
  select(date, starts_with("mom_")) %>%
```

```r
  arrange(date)

# ----------------------------------------------------------
# Join macro and momentum signals into V2 signal table
# Also create standardized momentum measures
# ----------------------------------------------------------
v2_signals <- v2_macro_signals %>%
  left_join(v2_momentum_signals, by = "date") %>%
  mutate(
    z_mom_us_equity   = roll_zscore(mom_us_equity_12_1, 36),
    z_mom_intl_equity = roll_zscore(mom_intl_equity_12_1, 36),
    z_mom_treasury    = roll_zscore(mom_treasury_12_1, 36),
    z_mom_credit      = roll_zscore(mom_credit_12_1, 36),
    z_mom_real_assets = roll_zscore(mom_real_assets_12_1, 36),

    # Broad risk appetite momentum signal
    z_mom_risk = rowMeans(
      across(c(z_mom_us_equity, z_mom_intl_equity, z_mom_credit)),
      na.rm = TRUE
    )
  ) %>%
  mutate(
    across(starts_with("z_mom_"), ~na_if(., NaN))
  ) %>%
  arrange(date)

# ----------------------------------------------------------
# Keep a compact working table for V2
# This is the clean input to V2 Step 2
# ----------------------------------------------------------
v2_signal_table <- v2_signals %>%
  select(
    date,

    # Inflation z-signals
    z_cpi_yoy,
    z_core_cpi_yoy,
    z_cpi_3m_ann,
    z_core_cpi_3m_ann,
    z_cpi_yoy_6m_delta,
    z_core_cpi_yoy_6m_delta,

    # Growth z-signals
    z_unrate_trend,
    z_payems_growth,
    z_indpro_growth,
    z_housing_growth,
    z_sentiment_change,

    # Policy / curve / credit z-signals
    z_real_fedfunds_risk,
    z_real_10y_risk,
    z_curve_10y3m,
```

```r
    z_curve_10y2y,
    z_curve_2y3m,
    z_baa_aaa_risk,
    z_baa_10y_risk,

    # Momentum
    z_mom_us_equity,
    z_mom_intl_equity,
    z_mom_treasury,
    z_mom_credit,
    z_mom_real_assets,
    z_mom_risk
  ) %>%
  arrange(date)

# ---------------------------------------------------------
# Inspect / save
# ---------------------------------------------------------
glimpse(v2_signal_table)
print(head(v2_signal_table, 15))
print(tail(v2_signal_table, 15))

write_csv(v2_signal_table, "v2_signal_table_step1.csv")

v2_signal_table_clean <- v2_signal_table %>%
  filter(if_all(-date, ~ !is.na(.))) %>%
  arrange(date)

glimpse(v2_signal_table_clean)

# =========================================================
# V2 Step 2: Build composite macro + momentum score
# Structure:
# - Equal weight within each block
# - Equal weight across blocks
# - Lag signals by 1 month for execution
# =========================================================

# ---------------------------------------------------------
# Use cleaned signal table from Step 1
# ---------------------------------------------------------
v2_step2 <- v2_signal_table_clean %>%
  arrange(date) %>%
  mutate(

    # ======================================================
    # 1. Inflation block (higher = hotter inflation)
    # ======================================================
    inflation_score = rowMeans(
      across(c(
        z_cpi_yoy,
        z_core_cpi_yoy,
        z_cpi_3m_ann,
```

```
      z_core_cpi_3m_ann,
      z_cpi_yoy_6m_delta,
      z_core_cpi_yoy_6m_delta
    )),
    na.rm = TRUE
  ),

  # ======================================================
  # 2. Growth block (higher = stronger growth)
  # ======================================================
  growth_score = rowMeans(
    across(c(
      z_unrate_trend,
      z_payems_growth,
      z_indpro_growth,
      z_housing_growth,
      z_sentiment_change
    )),
    na.rm = TRUE
  ),

  # ======================================================
  # 3. Policy / liquidity block (higher = easier conditions)
  # ======================================================
  policy_score = rowMeans(
    across(c(
      z_real_fedfunds_risk,
      z_real_10y_risk,
      z_curve_10y3m,
      z_curve_10y2y,
      z_curve_2y3m,
      z_baa_aaa_risk,
      z_baa_10y_risk
    )),
    na.rm = TRUE
  ),

  # ======================================================
  # 4. Risk / momentum block (higher = risk-on confirmation)
  # ======================================================
  risk_score = rowMeans(
    across(c(
      z_mom_us_equity,
      z_mom_intl_equity,
      z_mom_credit,
      z_mom_real_assets
    )),
    na.rm = TRUE
  )
) %>%
mutate(
  across(
    c(inflation_score, growth_score, policy_score, risk_score),
```

```r
      ~na_if(., NaN)
    )
  ) %>%
  mutate(

    # ====================================================
    # 5. Composite macro score
    # Equal weight across 4 blocks
    # ====================================================
    composite_score_raw = rowMeans(
      across(c(
        inflation_score,
        growth_score,
        policy_score,
        risk_score
      )),
      na.rm = TRUE
    )
  ) %>%
  mutate(

    # Optional normalization (keeps scale stable over time)
    composite_score_z = rollapplyr(
      composite_score_raw,
      width = 36,
      FUN = function(x) {
        s <- sd(x, na.rm = TRUE)
        m <- mean(x, na.rm = TRUE)
        last_x <- tail(x, 1)

        if (is.na(s) || s == 0) {
          NA_real_
        } else {
          (last_x - m) / s
        }
      },
      fill = NA,
      partial = FALSE
    )
  ) %>%
  mutate(

    # ====================================================
    # 6. Execution signal (lagged)
    # Prevents lookahead bias
    # ====================================================
    signal_for_allocation = lag(composite_score_z, 1)
  ) %>%
  arrange(date)

# ---------------------------------------------------------
# Clean final usable dataset for allocation
# ---------------------------------------------------------
```

```r
v2_step2_clean <- v2_step2 %>%
  filter(!is.na(signal_for_allocation)) %>%
  arrange(date)

# ---------------------------------------------------------
# Inspect
# ---------------------------------------------------------
glimpse(v2_step2_clean)

v2_step2_clean %>%
  select(
    date,
    inflation_score,
    growth_score,
    policy_score,
    risk_score,
    composite_score_z,
    signal_for_allocation
  ) %>%
  slice_head(n = 10)

v2_step2_clean %>%
  slice_tail(n = 10)

# ---------------------------------------------------------
# Save
# ---------------------------------------------------------
write_csv(v2_step2_clean, "v2_step2_scores.csv")

# =========================================================
# V2 Step 3: Map composite score -> dynamic portfolio weights
# Improved version:
# - rank-based scaling
# - convex allocation
# - momentum override
# - more dispersion between risk-on and risk-off states
# =========================================================

df <- v2_step2_clean %>%
  arrange(date)

clamp <- function(x, lower, upper) {
  pmin(pmax(x, lower), upper)
}

v2_weights <- df %>%
  arrange(date) %>%
  mutate(
    # -------------------------------------------------------
    # 1. Separate macro and momentum components
    # -------------------------------------------------------
    macro_score = rowMeans(
      across(c(inflation_score, growth_score, policy_score)),
```

```r
    na.rm = TRUE
  ),
  momentum_score = risk_score,

  # ----------------------------------------------------
  # 2. Rank-based scaling
  # Expands signal distribution to [0, 1]
  # ----------------------------------------------------
  macro_rank = percent_rank(macro_score),
  momentum_rank = percent_rank(momentum_score),

  # ----------------------------------------------------
  # 3. Combine macro + momentum
  # ----------------------------------------------------
  blended_rank = 0.3 * macro_rank + 0.7 * momentum_rank,

  # ----------------------------------------------------
  # 4. Convex mapping
  # ----------------------------------------------------
  risk_budget_base = if_else(
    blended_rank > 0.5,
    blended_rank^3,
    blended_rank^2 * 0.5
  ),

  # ----------------------------------------------------
  # 5. Initial momentum override using broad risk momentum
  # ----------------------------------------------------
  risk_budget_adj = case_when(
    z_mom_risk < -0.5 ~ risk_budget_base * 0.45,
    z_mom_risk <  0.0 ~ risk_budget_base * 0.70,
    TRUE              ~ risk_budget_base
  )
) %>%
mutate(
  # ----------------------------------------------------
  # 5b. Extra equity-trend override
  # ----------------------------------------------------
  risk_budget_adj = if_else(
    z_mom_us_equity < 0,
    risk_budget_adj * 0.4,
    risk_budget_adj
  ),

  # ----------------------------------------------------
  # 6. Floor / ceiling
  # ----------------------------------------------------
  w_risk_total = clamp(risk_budget_adj, 0.15, 0.95),
  w_def_total  = 1 - w_risk_total
) %>%
mutate(
  # ----------------------------------------------------
  # 7. Split within risk bucket
```

```r
  # Make composition responsive to inflation and momentum
  # -----------------------------------------------------

  # Equity share inside risk bucket
  equity_share_raw = 0.70 + 0.25 * momentum_rank - 0.10 * pmax(inflation_score, 0),
  equity_share = clamp(equity_share_raw, 0.35, 0.75),

  # Credit share inside risk bucket
  credit_share_raw = 0.12 + 0.03 * momentum_rank,
  credit_share = clamp(credit_share_raw, 0.10, 0.30),

  # Real asset share inside risk bucket
  real_asset_share_raw = 1 - equity_share - credit_share,
  real_asset_share = clamp(real_asset_share_raw, 0.10, 0.35),

  # Renormalize risk sleeves
  risk_share_sum = equity_share + credit_share + real_asset_share,

  equity_share_n = equity_share / risk_share_sum,
  credit_share_n = credit_share / risk_share_sum,
  real_asset_share_n = real_asset_share / risk_share_sum
) %>%
mutate(
  # -----------------------------------------------------
  # 8. Split equity sleeve into US / Intl
  # -----------------------------------------------------
  us_share_raw = 0.70 + 0.10 * momentum_rank,
  us_share = clamp(us_share_raw, 0.60, 0.85),
  intl_share = 1 - us_share
) %>%
mutate(
  # -----------------------------------------------------
  # 9. Split defensive sleeve into Treasuries / Cash
  # -----------------------------------------------------
  treasury_share_raw = 0.75 + 0.10 * (1 - momentum_rank),
  treasury_share = clamp(treasury_share_raw, 0.70, 0.90),
  cash_share = 1 - treasury_share
) %>%
mutate(
  # --------------------
  w_equity_total = w_risk_total * equity_share_n,

  w_credit       = w_risk_total * credit_share_n,
  w_real_assets  = w_risk_total * real_asset_share_n,

  w_us_equity    = w_equity_total * us_share,
  w_intl_equity  = w_equity_total * intl_share,

  w_treasury     = w_def_total * treasury_share,
  w_cash         = w_def_total * cash_share
) %>%
mutate(
  # -----------------------------------------------------
```

```r
    # 11. Check sum
    # -------------------------------------------------------
    w_total =
      w_us_equity +
      w_intl_equity +
      w_credit +
      w_real_assets +
      w_treasury +
      w_cash
  ) %>%
  arrange(date)

# Checks
v2_weights %>%
  summarise(
    min_total = min(w_total, na.rm = TRUE),
    max_total = max(w_total, na.rm = TRUE)
  )

v2_weights %>%
  summarise(
    min_risk = min(w_risk_total, na.rm = TRUE),
    p25_risk = quantile(w_risk_total, 0.25, na.rm = TRUE),
    median_risk = median(w_risk_total, na.rm = TRUE),
    p75_risk = quantile(w_risk_total, 0.75, na.rm = TRUE),
    max_risk = max(w_risk_total, na.rm = TRUE)
  )

v2_weights %>%
  select(
    date,
    macro_score,
    momentum_score,
    blended_rank,
    z_mom_risk,
    w_risk_total,
    w_us_equity,
    w_intl_equity,
    w_credit,
    w_real_assets,
    w_treasury,
    w_cash
  ) %>%
  slice_head(n = 10)

write_csv(v2_weights, "v2_weights_step3_improved.csv")

# ========================================================
# V2 Step 4: Compute portfolio returns
# Improved version:
# - uses improved sleeve weights
# - computes gross and net returns
# - same benchmark structure can be reused in Step 5
```

```r
# ==========================================================

# ----------------------------------------------------------
# 1. Map sleeve returns from V1 asset_returns
# ----------------------------------------------------------
v2_returns_base <- asset_returns %>%
  arrange(date) %>%
  transmute(
    date,

    # US equity sleeve
    us_equity =
      0.50 * spy_ret +
      0.30 * qqq_ret +
      0.20 * iwm_ret,

    # International equity sleeve
    intl_equity =
      0.60 * efa_ret +
      0.40 * eem_ret,

    # Credit sleeve
    credit =
      0.60 * lqd_ret +
      0.40 * hyg_ret,

    # Real asset sleeve
    real_assets =
      0.25 * tip_ret +
      0.25 * gld_ret +
      0.25 * dbc_ret +
      0.25 * vnq_ret,

    # Treasury sleeve
    treasury =
      0.50 * ief_ret +
      0.50 * tlt_ret,

    # Cash sleeve
    cash = bil_ret
  )

# ----------------------------------------------------------
# 2. Join weights with returns
# ----------------------------------------------------------
v2_portfolio <- v2_weights %>%
  select(
    date,
    w_us_equity,
    w_intl_equity,
    w_credit,
    w_real_assets,
    w_treasury,
```

```r
    w_cash
  ) %>%
  left_join(v2_returns_base, by = "date") %>%
  arrange(date)

# ---------------------------------------------------------
# 3. Compute gross portfolio return
# ---------------------------------------------------------
v2_portfolio <- v2_portfolio %>%
  mutate(
    port_ret_gross =
      w_us_equity   * us_equity +
      w_intl_equity * intl_equity +
      w_credit      * credit +
      w_real_assets * real_assets +
      w_treasury    * treasury +
      w_cash        * cash
  )

# ---------------------------------------------------------
# 4. Transaction costs
# Use half-turnover convention to avoid double counting
# ---------------------------------------------------------
tc_per_turnover <- 0.001

v2_portfolio <- v2_portfolio %>%
  arrange(date) %>%
  mutate(
    turnover =
      0.5 * (
        abs(w_us_equity   - lag(w_us_equity)) +
        abs(w_intl_equity - lag(w_intl_equity)) +
        abs(w_credit      - lag(w_credit)) +
        abs(w_real_assets - lag(w_real_assets)) +
        abs(w_treasury    - lag(w_treasury)) +
        abs(w_cash        - lag(w_cash))
      ),
    turnover = replace_na(turnover, 0),
    tc_cost = turnover * tc_per_turnover,
    port_ret_net = port_ret_gross - tc_cost
  )

# ---------------------------------------------------------
# 5. Clean final dataset
# ---------------------------------------------------------
v2_portfolio_clean <- v2_portfolio %>%
  filter(!is.na(port_ret_net)) %>%
  arrange(date)

# ---------------------------------------------------------
# 6. Inspect / save
# ---------------------------------------------------------
glimpse(v2_portfolio_clean)
```

```r
v2_portfolio_clean %>%
  select(date, port_ret_gross, port_ret_net, turnover, tc_cost) %>%
  slice_head(n = 10)

v2_portfolio_clean %>%
  slice_tail(n = 10)

write_csv(v2_portfolio_clean, "v2_portfolio_returns_improved.csv")

# ==========================================================
# V2 Step 5: Performance evaluation
# Structure:
# - Compute cumulative returns
# - Risk/return metrics
# - Benchmark comparison (SPY, 60/40, All Weather)
# ==========================================================

suppressPackageStartupMessages({
  library(tidyverse)
  library(lubridate)
})

# ----------------------------------------------------------
# 1. Prepare strategy returns
# ----------------------------------------------------------
strategy <- v2_portfolio_clean %>%
  select(date, port_ret_gross, port_ret_net) %>%
  arrange(date)

# ----------------------------------------------------------
# 2. Build benchmark returns (same as V1)
# ----------------------------------------------------------
benchmarks <- asset_returns %>%
  arrange(date) %>%
  transmute(
    date,

    # SPY
    SPY = spy_ret,

    # 60/40 (stocks/bonds)
    `60_40` =
      0.6 * spy_ret +
      0.4 * (0.5 * ief_ret + 0.5 * tlt_ret),

    # All Weather (simple proxy)
    AllWeather =
      0.3 * spy_ret +
      0.4 * (0.5 * ief_ret + 0.5 * tlt_ret) +
      0.15 * gld_ret +
      0.15 * dbc_ret
  )
```

```r
# ---------------------------------------------------------
# 3. Merge
# ---------------------------------------------------------
perf_df <- strategy %>%
  left_join(benchmarks, by = "date") %>%
  drop_na()

# ---------------------------------------------------------
# 4. Cumulative returns
# ---------------------------------------------------------
perf_df <- perf_df %>%
  mutate(
    strat_gross_cum = cumprod(1 + port_ret_gross),
    strat_net_cum   = cumprod(1 + port_ret_net),
    spy_cum         = cumprod(1 + SPY),
    bal6040_cum     = cumprod(1 + `60_40`),
    aw_cum          = cumprod(1 + AllWeather)
  )

# ---------------------------------------------------------
# 5. Performance metrics function
# ---------------------------------------------------------
perf_stats <- function(ret) {
  n <- length(ret)
  ann_ret <- prod(1 + ret)^(12 / n) - 1
  ann_vol <- sd(ret) * sqrt(12)

  sharpe <- ifelse(ann_vol == 0, NA, ann_ret / ann_vol)

  downside <- sd(ret[ret < 0]) * sqrt(12)
  sortino <- ifelse(downside == 0, NA, ann_ret / downside)

  cum <- cumprod(1 + ret)
  drawdown <- cum / cummax(cum) - 1
  max_dd <- min(drawdown, na.rm = TRUE)

  calmar <- ifelse(max_dd == 0, NA, ann_ret / abs(max_dd))

  hit_rate <- mean(ret > 0)

  tibble(
    Months = n,
    CAGR = ann_ret,
    AnnVol = ann_vol,
    Sharpe = sharpe,
    Sortino = sortino,
    Calmar = calmar,
    MaxDrawdown = max_dd,
    HitRate = hit_rate,
    BestMonth = max(ret, na.rm = TRUE),
    WorstMonth = min(ret, na.rm = TRUE)
  )
}
```

```r
# ---------------------------------------------------------
# 6. Compute metrics
# ---------------------------------------------------------
v2_summary <- bind_rows(
  perf_stats(perf_df$port_ret_gross) %>% mutate(series = "V2_Strategy_Gross"),
  perf_stats(perf_df$port_ret_net)   %>% mutate(series = "V2_Strategy_Net"),
  perf_stats(perf_df$SPY)            %>% mutate(series = "SPY"),
  perf_stats(perf_df$`60_40`)        %>% mutate(series = "60_40"),
  perf_stats(perf_df$AllWeather)     %>% mutate(series = "AllWeather")
) %>%
  select(series, everything())

# ---------------------------------------------------------
# 7. Print results
# ---------------------------------------------------------
print(v2_summary)

# ---------------------------------------------------------
# 8. Optional: Save cumulative series for charting
# ---------------------------------------------------------
cum_df <- perf_df %>%
  select(
    date,
    strat_gross_cum,
    strat_net_cum,
    spy_cum,
    bal6040_cum,
    aw_cum
  )

write_csv(cum_df, "v2_cumulative_returns.csv")
write_csv(v2_summary, "v2_performance_summary.csv")
```